CSE 554- Machine Learning Systems
Lecture 16 - RL for LLMs

Baris Kasikci

Class Overview

1. GPU Architecture and Programming
2. Transformers (Architecture and Implementation)
3. Performance Modeling
4. Quantization
5. Memory Management
6. NVIDIA Blackwell Architecture
7. Sparsity
8. Mixture of Experts (MoE)
9. Speculative decoding
10. Batching
11. Parallelism
12. **RL for LLMs**
13. Agentic systems

Contents

- RL Basics
  - Example of a Grid World
    - Key Concepts
    - RL Algorithms
    - One Simulation
  - Formal Summary
- RL for LLMs
  - Motivation
  - Conventional LLM Training
  - Role of LLM in RL
  - PPO & GRPO
  - RLHF to RLVR
  - Reasoning
- RL Systems
  - Programing model
  - Recent research

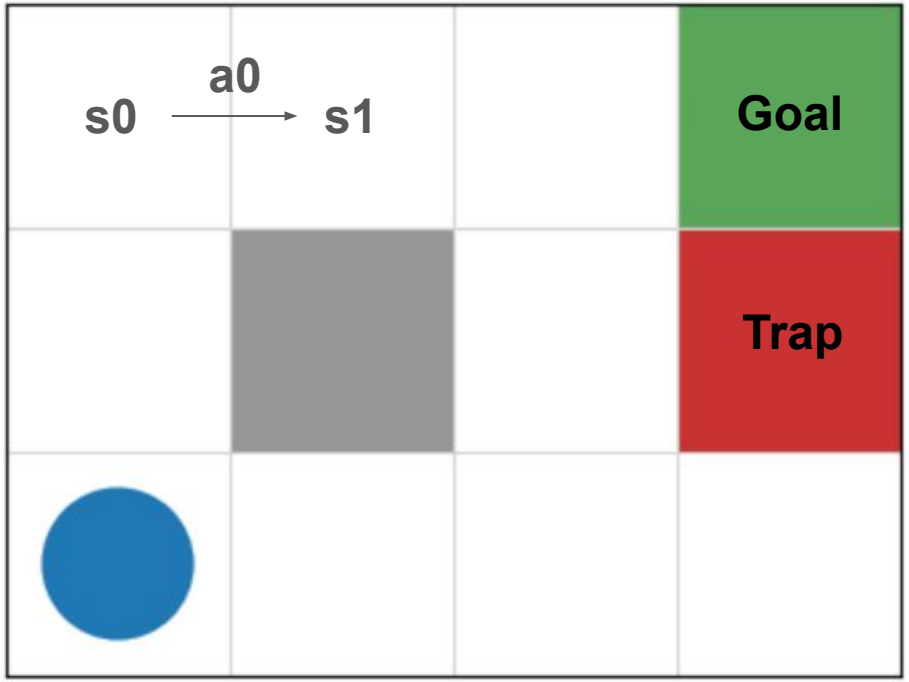
RL Basics: Grid World

An **agent** is at the bottom left cell of a grid. The gray cell is a wall.

https://gibberblot.github.io/rl-notes/single-agent/MDPs.html#

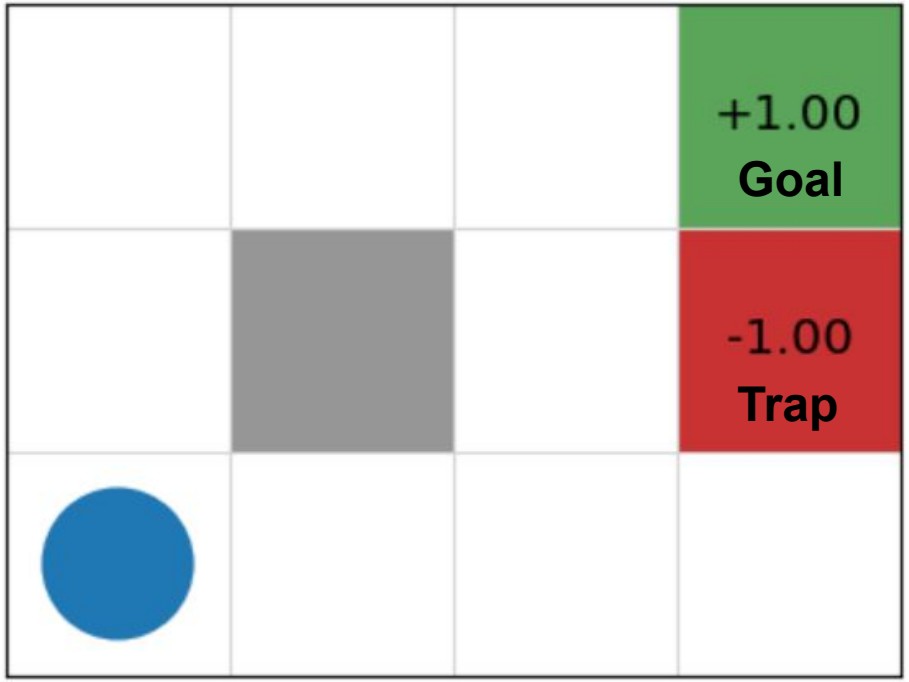
RL Basics: Key Concepts

- State **s**: Description of the world
- Action **a**: changes the state: moves from one cell to another cell



RL Basis: Key Concepts

- Policy π: determines which action should the agent take at any state
- Reward **r**: a scalar feedback the agent receives from the environment after taking an action at a state
- **Return** is the cumulative sum of rewards an agent receives starting in a state s onward
- Value **V**: expected return if you start in a state or state-action pair, and then act according to a particular policy thereafter



RL Basics: RL Algorithms

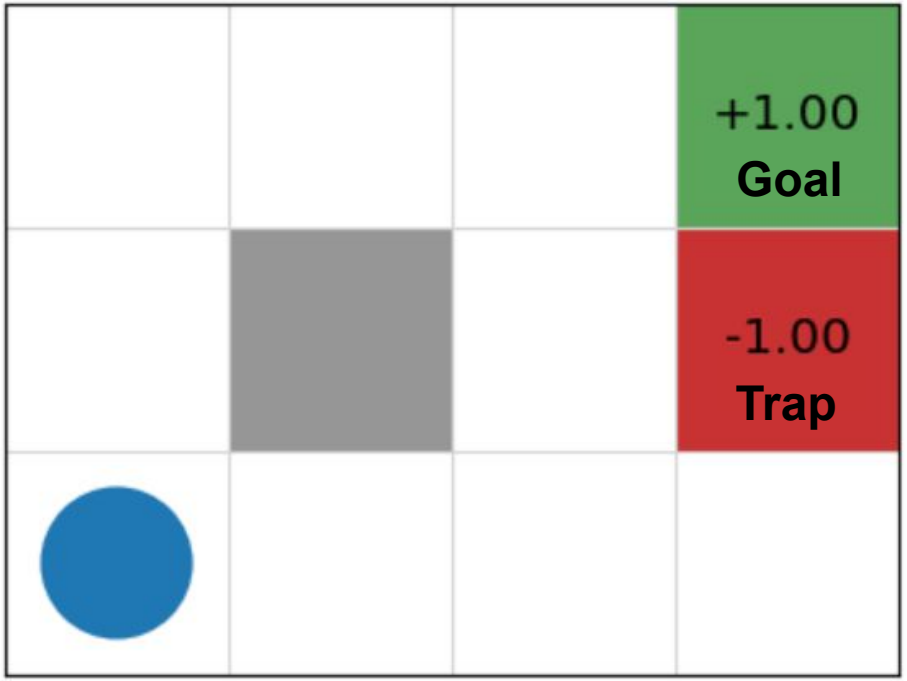
- Value-Based Methods (we'll give the intuition of this one)
- Policy-Based Methods
- Actor-Critic Methods

RL is a vast topic of research, out of scope for this class

RL Basics: How does a simple RL algorithm work?

Here we use **Value Iteration** as the RL algorithm.

1. Set the reward for each grid cell, +1 at goal, -1 at trap, 0 at other places



RL Basis: How to run a simple RL algorithm?

Here we use **Value Iteration** as the RL algorithm.

1. Set the reward for each grids, +1 at goal, -1 at trap, 0 at other places
2. Run **Value Iteration** to get the estimated value at each grid
   - The idea is to compute the expected value of success when a particular path is followed and update cell values accordingly

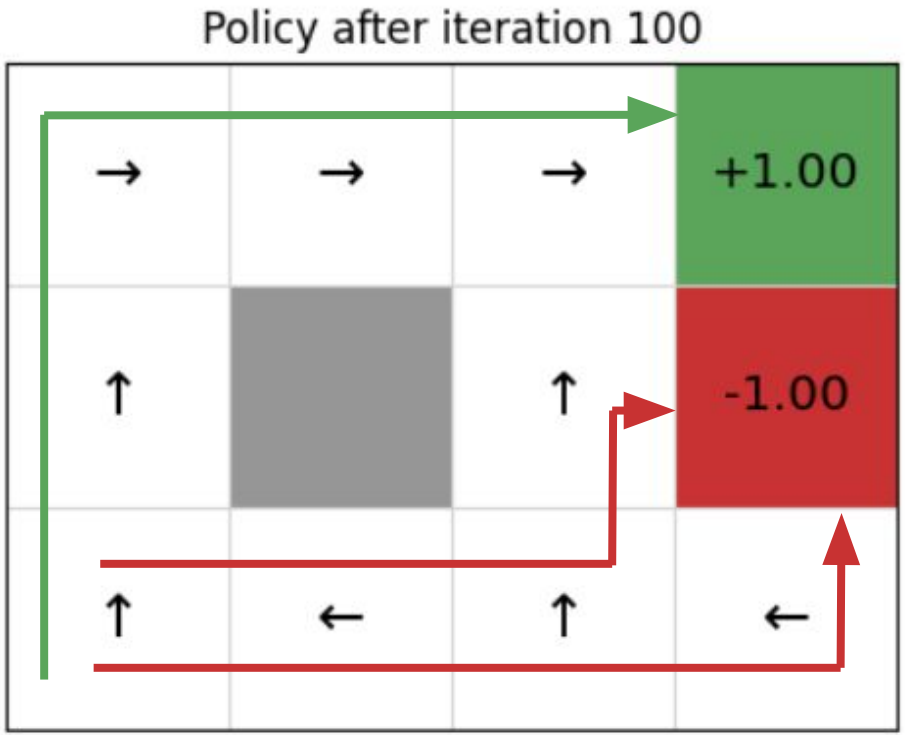
Value function after iteration 100

+0.64	+0.74	+0.85	+1.00
+0.57		+0.57	-1.00
+0.49	+0.43	+0.48	+0.28

RL Basics: How to run a simple RL algorithm?

Here we use **Value Iteration** as the RL algorithm.

1. Set the reward for each grids, +1 at goal, -1 at trap, 0 at other places.
2. Run **Value Iteration** to get the estimated value at each grid.
3. Extract the policy from the value distribution.

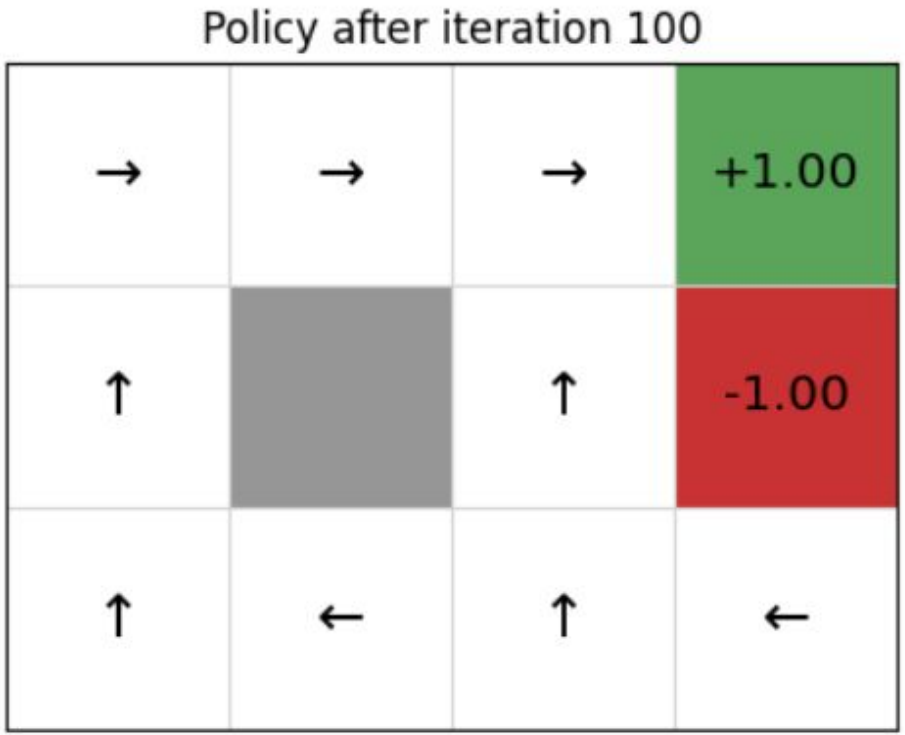


More of the red paths lead to an undesired outcome

RL Basics: How to run a simple RL algorithm?

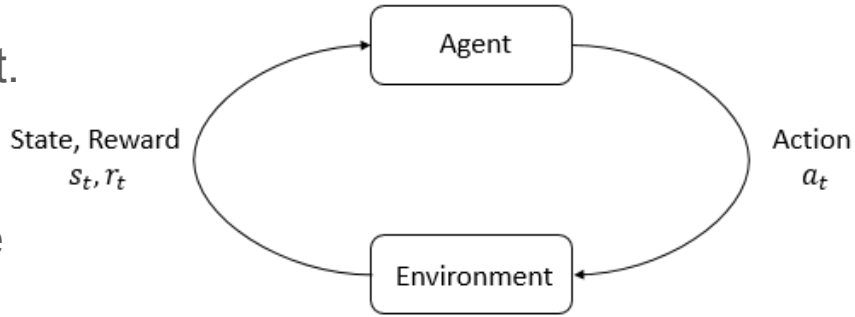
Here we use **Value Iteration** as the RL algorithm.

1. Set the reward for each grids, +1 at goal, -1 at trap, 0 at other places.
2. Run **Value Iteration** to get the estimated value at each grid.
3. Extract the policy from the value distribution.
4. Done! We just got a policy for this game!



RL Basics: Formal Summary

- An ML paradigm, where*:
  - An **agent** learns to make decisions based on **actions** in an environment.
    - Changes the **states** (i.e. Descriptions of the world)
  - The goal of the agent is to maximize **return** (sum of **rewards**)
- Agents use **policies** to take actions
  - Actions are sampled from policies (may not be deterministic)
- Policies are computable
  - Their outputs depend on parameters (weights, biases)
  - In our case, LLMs

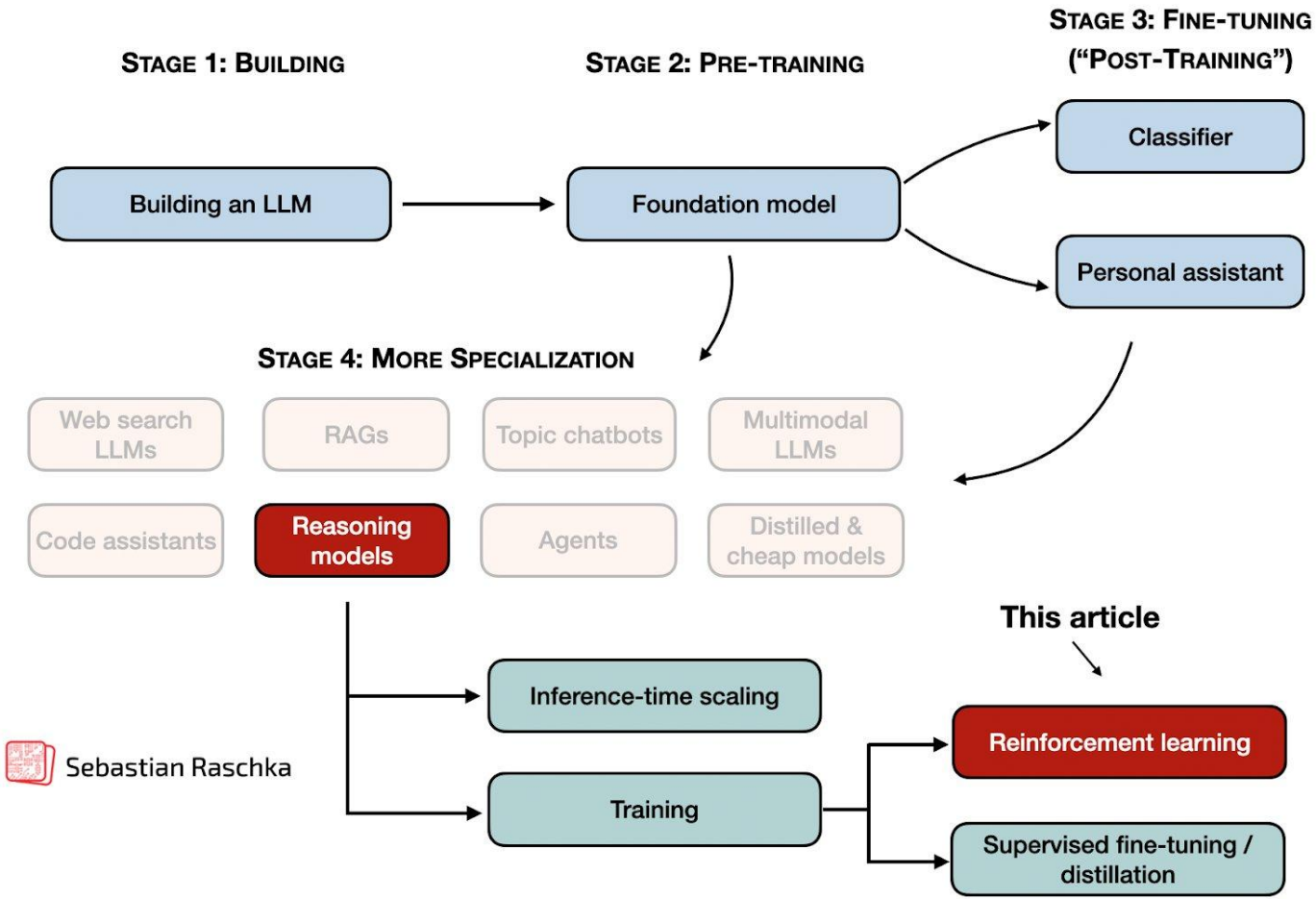


https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

RL for LLMs: Motivation

- There is significant room to improve LLMs via RL
  - We are approaching data limits
  - OpenAI reported that o3 used 10x more training compute compared to o1
- Reasonable to expect reasoning-focused post-training to become standard practice in future LLM pipelines.

Scaling RL: “more than 10x the training compute than o1”


Stage 1: Building
Stage 2: Pre-training
Stage 3: Fine-tuning ("Post-Training")
Building an LLM
Foundation model
Classifier
Personal assistant
Stage 4: More Specialization
Web search LLMs
RAGs
Topic chatbots
Multimodal LLMs
Code assistants
Reasoning models
Agents
Distilled & cheap models
This article
Inference-time scaling
Training
Reinforcement learning
Supervised fine-tuning / distillation
Sebastian Raschka

RL for LLMs: Conventional LLM Training

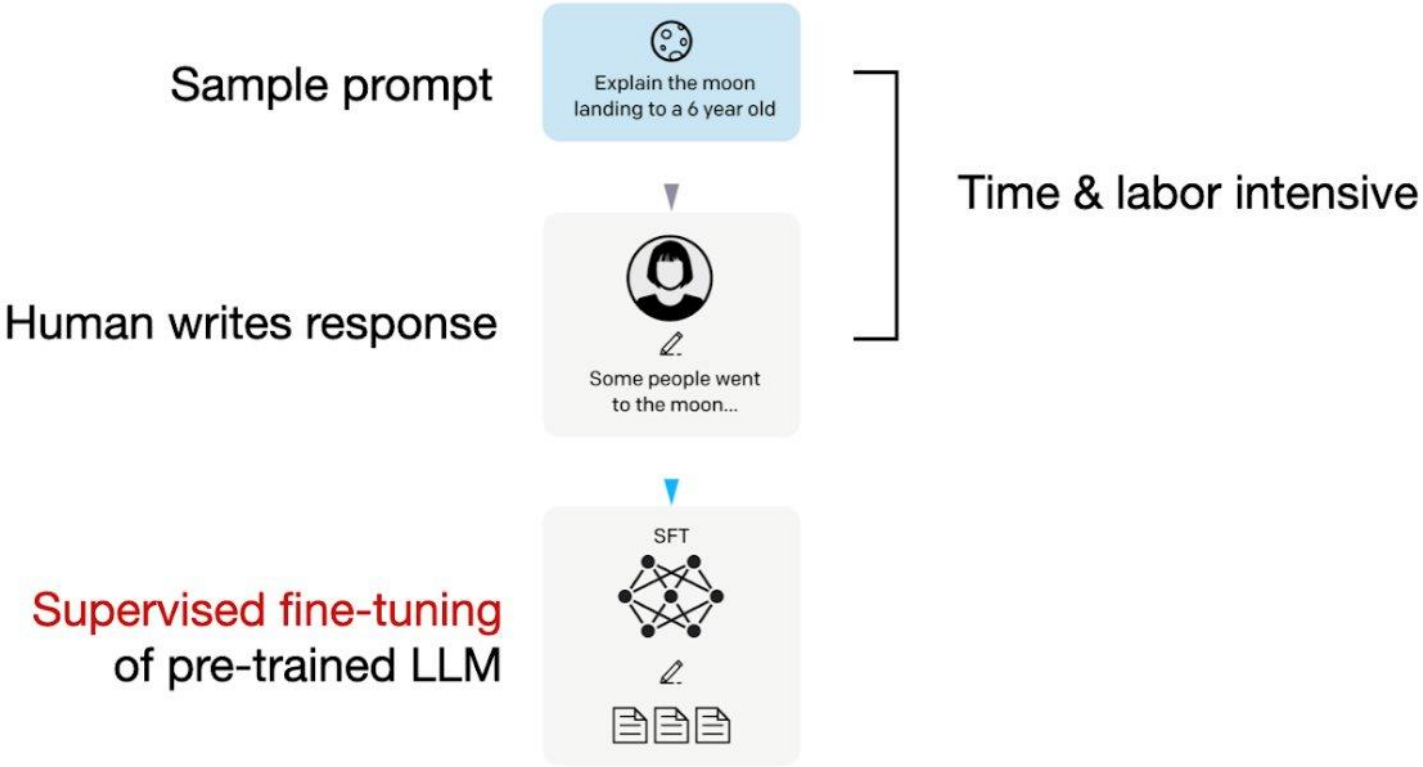
The recipe from InstructGPT (The ChatGPT paper: https://arxiv.org/pdf/2203.02155)

1. Pre-training
2. Supervised fine-tuning
3. Alignment (typically via RLHF)

The RLHF pipeline takes a pre-trained model and fine-tunes it in a supervised fashion.

Then, RLHF further aligns the LLM using an algorithm called proximal policy optimization (PPO).

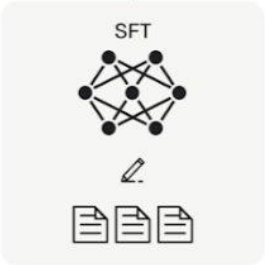
Supervised Fine-Tuning (Step 1: Prereq for RL Training)

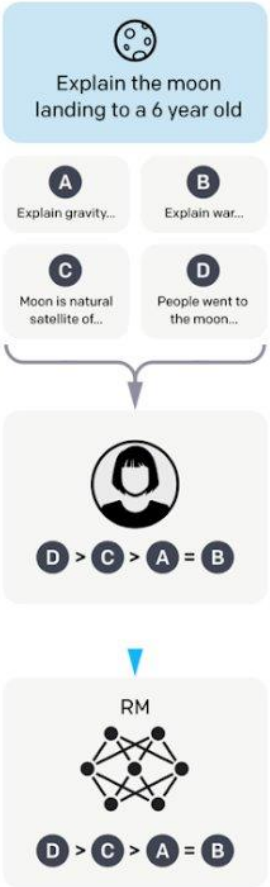


https://sebastianraschka.com/blog/2025/the-state-of-reinforcement-learning-for-llm-reasoning.html

RLHF Step 2

LLM fine-tuned in step 1:





Sample prompt

Collect model responses

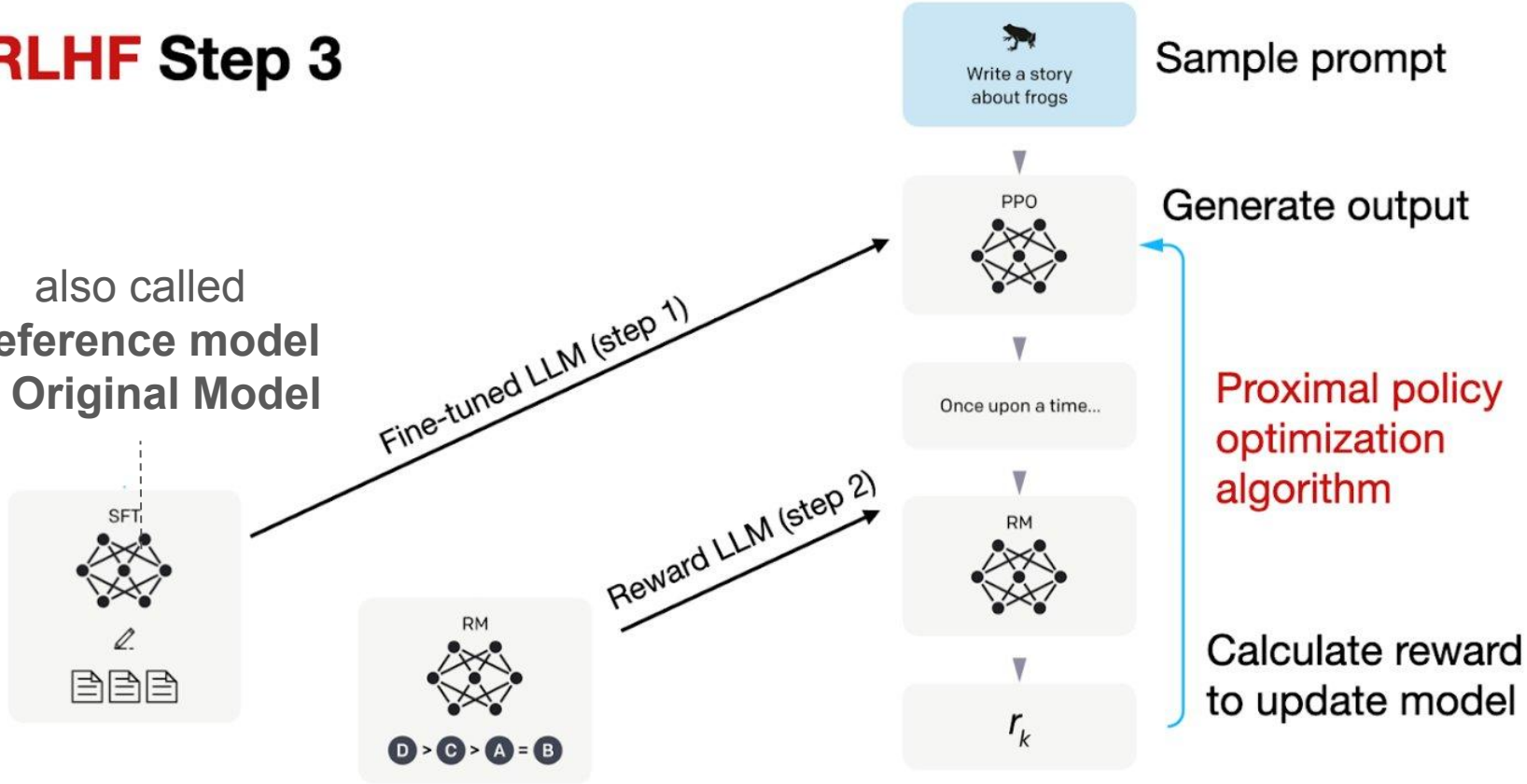
Human ranks responses

Time & labor intensive

Train reward model (Another LLM)

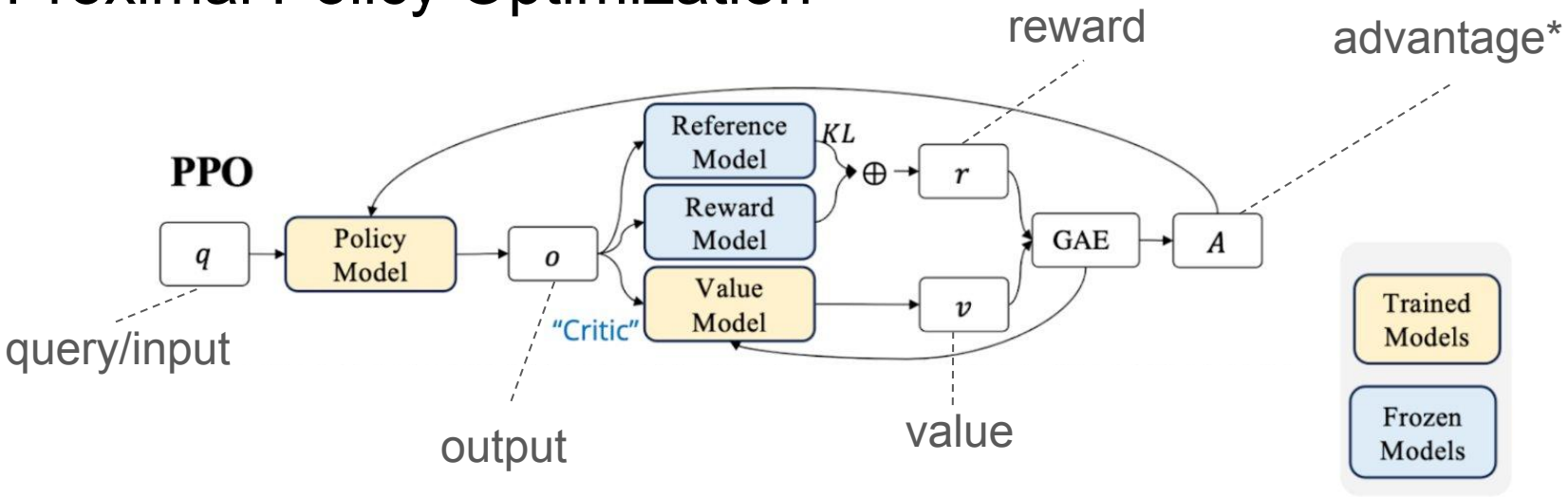
Ranking is easier than drafting responses, provides foundation for the rewards

RLHF Step 3



Use the reward model to fine tune the previous model from SFT

Proximal Policy Optimization

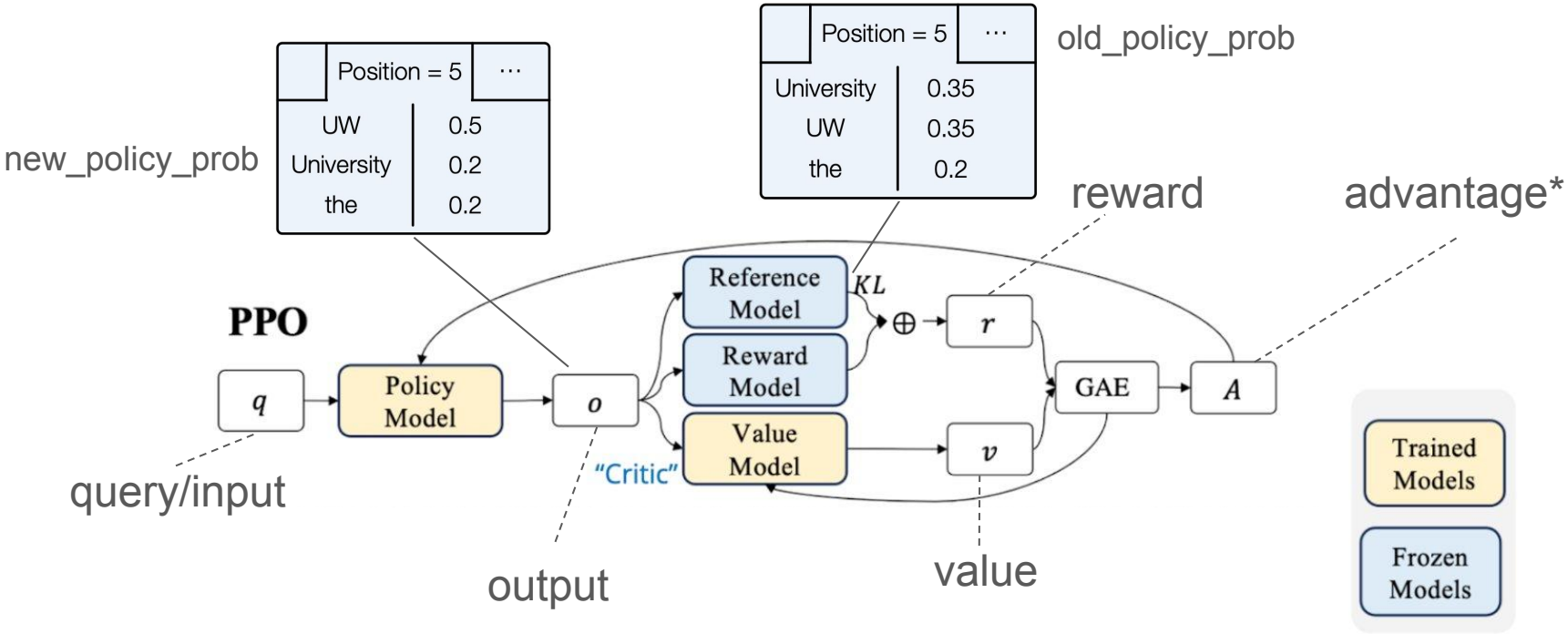


**advantage*: a quantity that tells how much better/worse an action is compared to the average action at that state

** GAE: Generalized advantage estimation: an advantage estimation algorithm

Critic/Value model: Another trainable model that estimates the expected reward

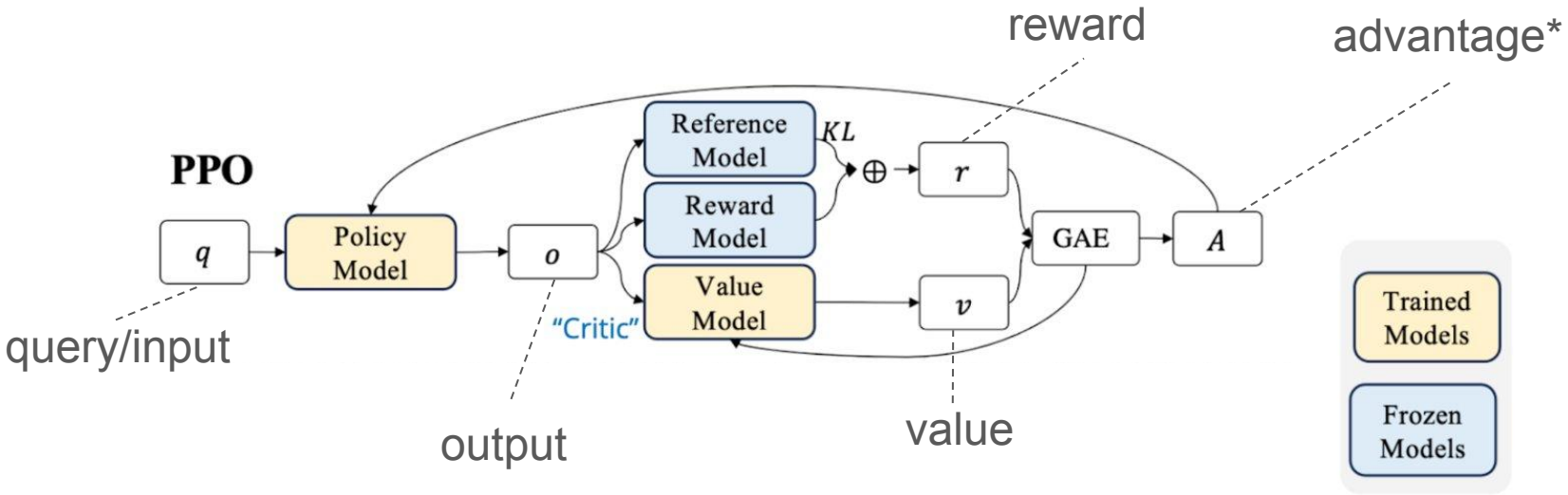
KL: measures the divergence between the old policy and the new policy at each state



Step - 1: Compute the ratio of the next-token probabilities from the new (policy model) vs the old policy (reference model):

```
ratio = new_policy_prob / old_policy_prob
```

Proximal Policy Optimization - Score Computation



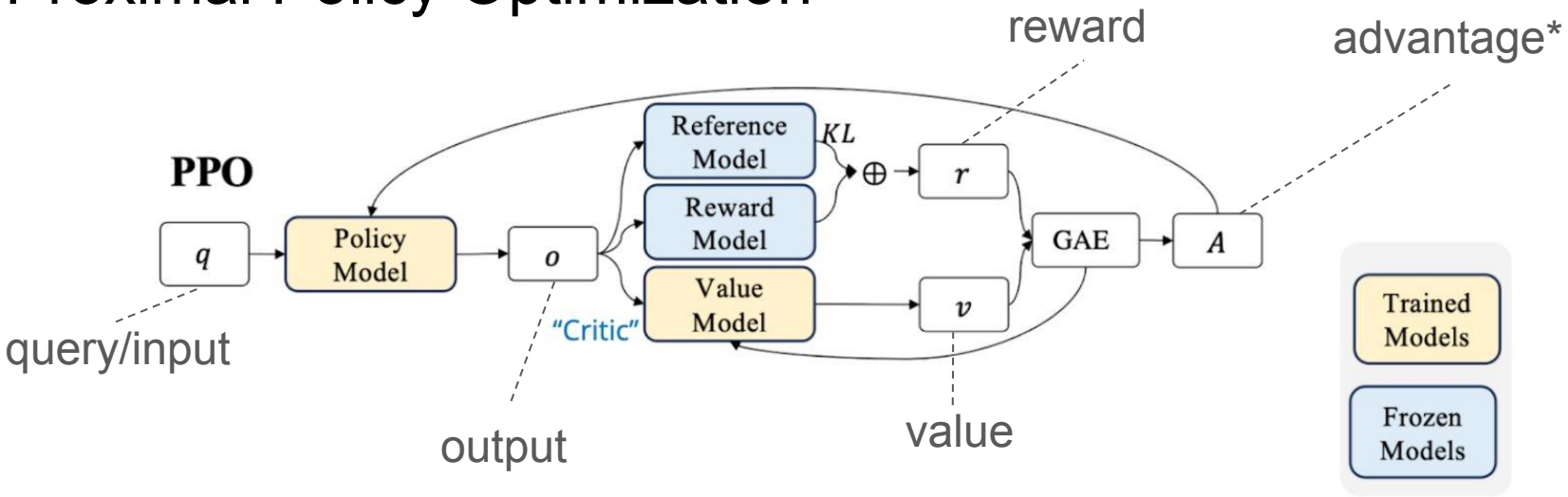
Step - 2: Multiply that ratio with the advantage, i.e., how good the action was:

`raw_score = ratio * advantage`

`clipped_score = clipped_ratio * advantage` (where `clipped_ratio = clamp(ratio, 0.8, 1.2)`)

Assume: `advantage = actual_reward - expected_reward`, critic/value model computes the `expected reward`

Proximal Policy Optimization



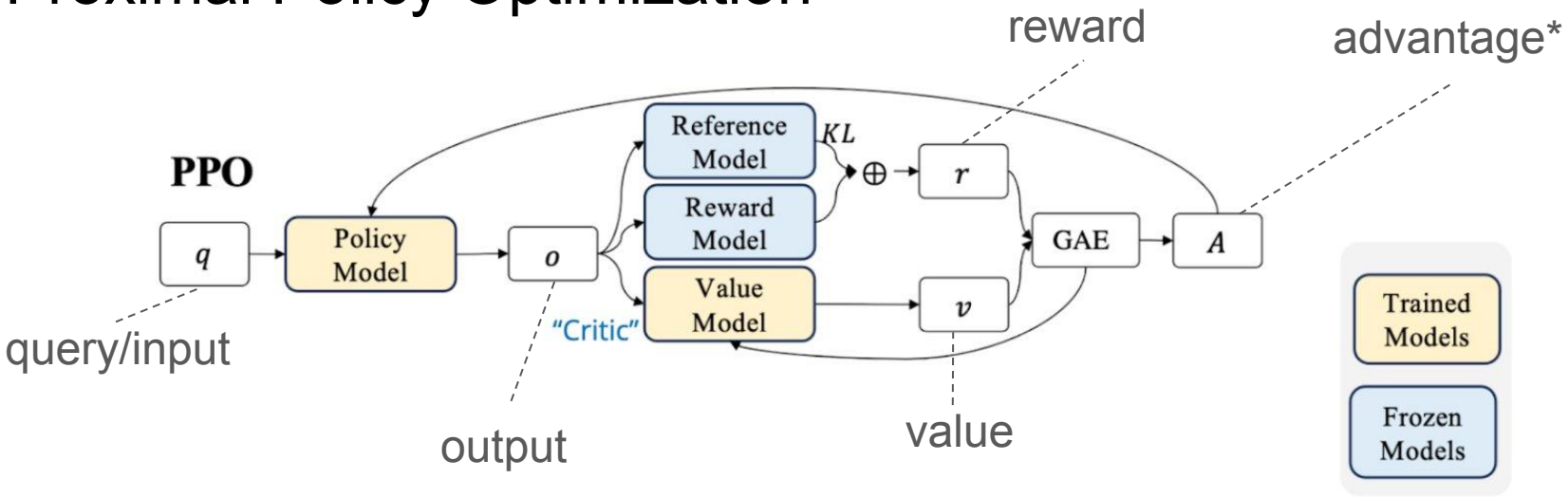
Step - 2: Multiply that ratio with the advantage, i.e., how good the action was:

```
raw_score = ratio * advantage
```

`clipped_score = clipped_ratio * advantage` (where `clipped_ratio = clamp(ratio, 0.8, 1.2)`)

```
final_score = min(raw_score, clipped_score)
```

Proximal Policy Optimization

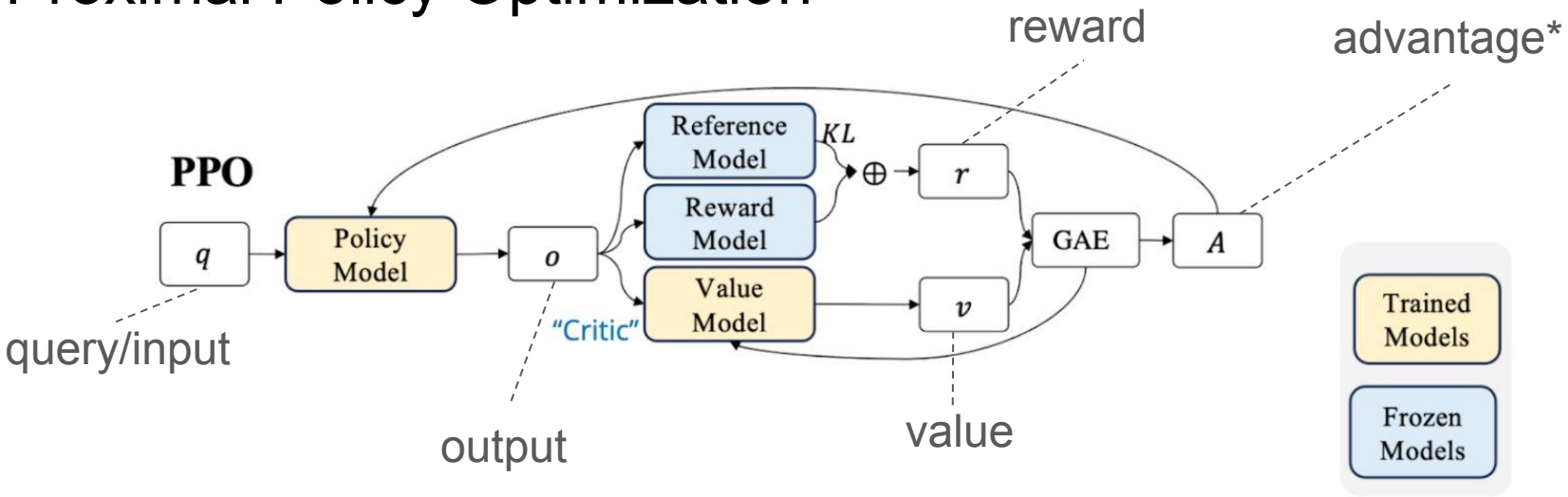


Step - 3: Loss function of the actor/policy model:

```
loss_actor = -final_score + □ * KL(new_policy || reference_policy)
```

KL-divergence will penalize if the divergence from the original model is too high

Proximal Policy Optimization

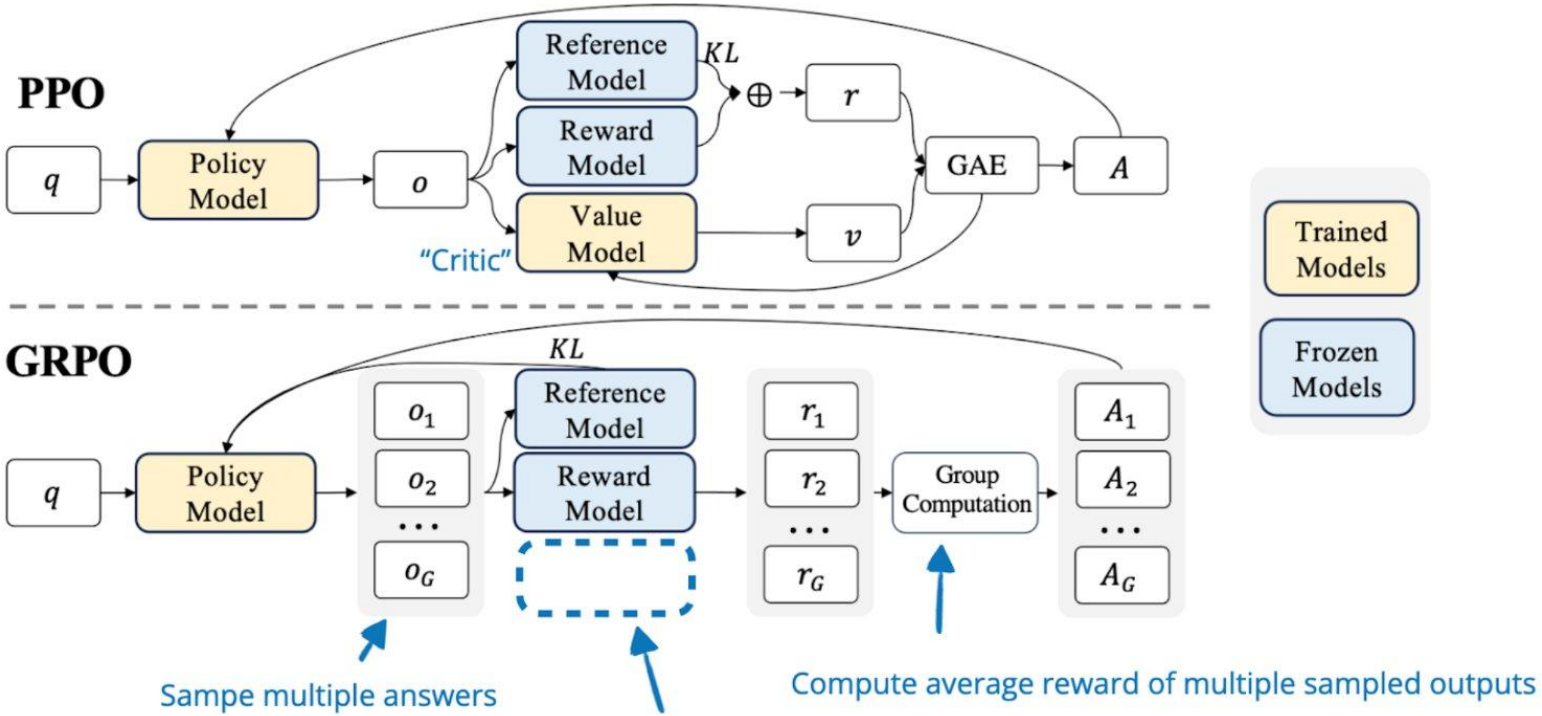


Step - 4: Loss function for the critic/value model:

```
loss_critic = mean_square error between v_old and v_target

where v_target = advantage + v_old.
```

GRPO (Group Relative PO - DeepSeekMath)



GRPO is more efficient as it forgoes the critic

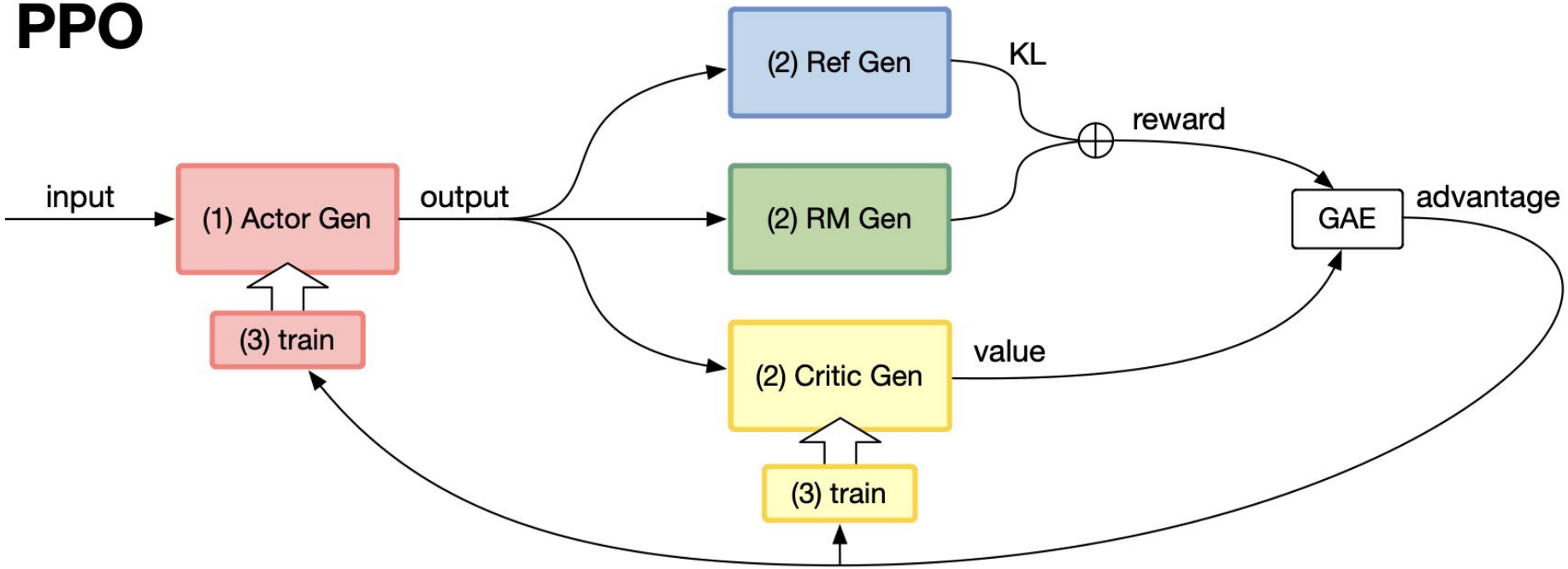
RL for LLMs: RLHF to RLVR and Reasoning Models

- Reasoning: model's ability to produce intermediate steps before providing a final answer
  - Chain-of-thought
- RLHF is already part of the LLM alignment toolkit
  - What's the connection to reasoning? Deepseek used a different approach than RLHF
    - Alternative: use verified reward insteading of human feedback
- Rather than learning what counts as a "good" from human-labeled examples, the model gets binary feedback (correct or wrong) from a tool, such as symbolic verifier or rule-based tools (e.g. a calculator)
  - In contrast to standard RLHF, RLVR bypasses the need for a reward model
  - RLVR is **much more efficient**
    - Tools are cheaper, and there is no reward model either in this case

RL for LLMs: Reasoning

- Reasoning, in the context of LLMs, refers to the model's ability to produce intermediate steps before providing a final answer. This is a process that is often described as chain-of-thought (CoT) reasoning. In CoT reasoning, the LLM explicitly generates a structured sequence of statements or computations that illustrate how it arrives at its conclusion.
- Sometimes referred to as chain of thought
- Perhaps the conclusion is that RL definitely turns simple base models into reasoning models. However, it's not the only way to induce or improve reasoning abilities.

PPO


input
(1) Actor Gen
output
(3) train
(2) Ref Gen
(2) RM Gen
(2) Critic Gen
(3) train
KL
reward
value
GAE
advantage

RL for LLMs: PPO Steps

- **Compute the ratio of the next-token probabilities from the new vs the old policy:**
  - Ratio = new_policy_prob / old_policy_prob
- **Multiply that ratio by how good the action was (called the advantage):**
  - Raw_score = ratio * advantage
  - Here for simplicity, we may assume the advantage is computed based on the reward signal:
    - Advantage = actual_reward - expected_reward
    - the **expected reward** is computed by a so-called “critic” (sometimes also called “value model”), and a **reward model computes the actual reward**. I.e., the advantage computation involves 2 other models, typically the same size as the original model we are fine-tuning.
- **Clipped_ratio = clamp(ratio, 0.8, 1.2) clipped_score = clipped_ratio * advantage.**
- **Then we use the smaller of the raw_score and clipped_score as the final_score**
- **Loss = -final_score + □ * KL(new_policy || reference_policy)**

RL for LLMs: PPO Summary

- The policy (actor model, agent), which is the LLM that has been trained with SFT and that we want to further align.
- The reward model, which is a model that has been trained to predict the reward (see RLHF step 2).
- The critic, which is a tranable model that estimates the reward.
- A reference model (original policy) that we use to make sure that the policy doesn’t deviate too much.

RLHF with **PPO**

Original policy
(LLM from SFT stage)

New policy
(LLM being trained with PPO)

Critic
(Value model estimating expected reward)

Reward model
(Trained from human feedback; predicts reward scores)

RLHF with **GRPO**

Original policy
(LLM from SFT stage)

New policy
(LLM being trained with PPO)

~~**Critic**~~
~~(Value model estimating expected reward)~~

Reward model
(Trained from human feedback; predicts reward scores)

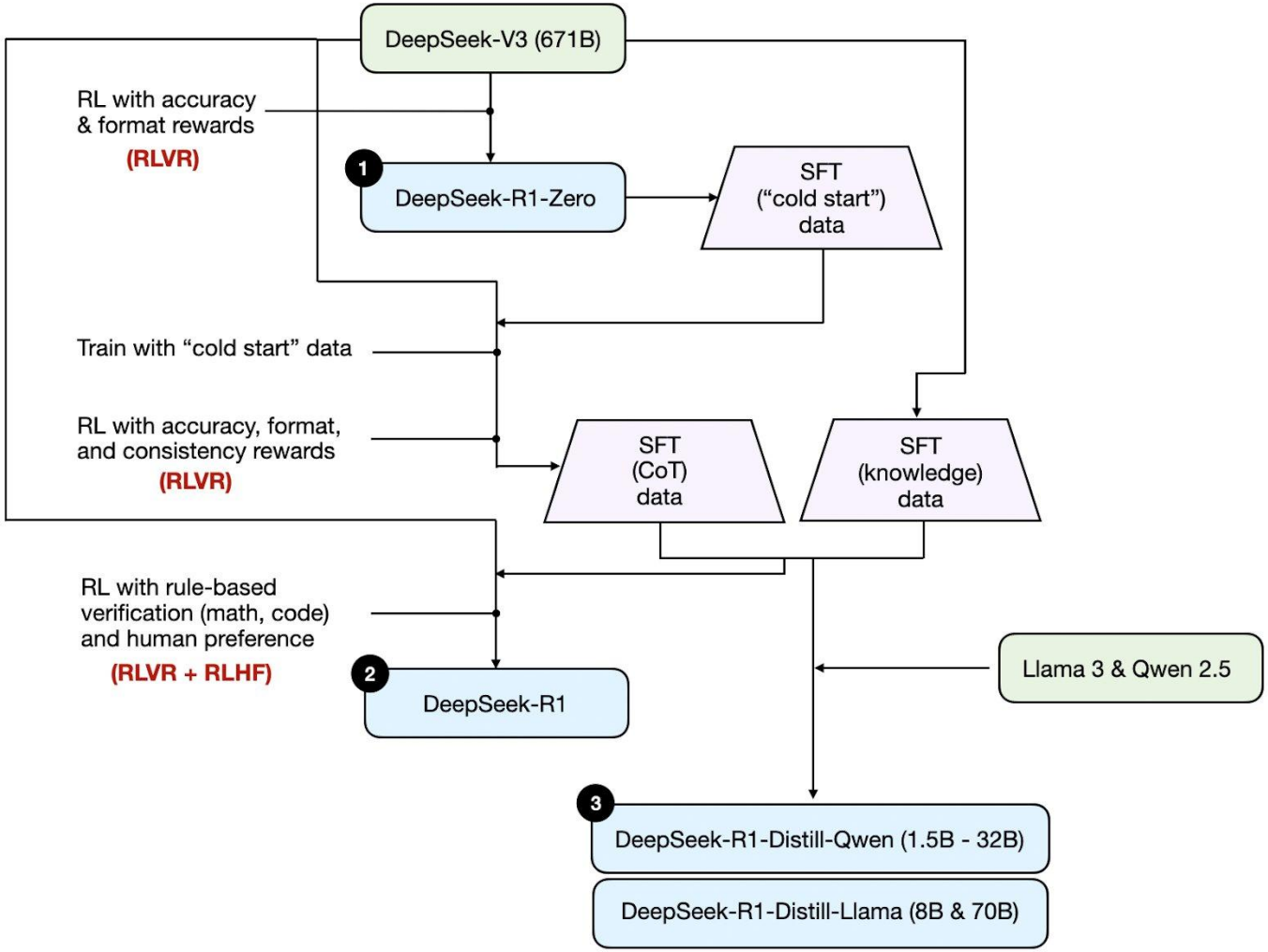
RLVR with GRPO

Original policy
(LLM from SFT stage)

New policy
(LLM being trained with PPO)

~~**Critic**~~
~~(Value model estimating expected reward)~~

~~**Reward model**~~
~~(Trained from human feedback; predicts reward scores)~~

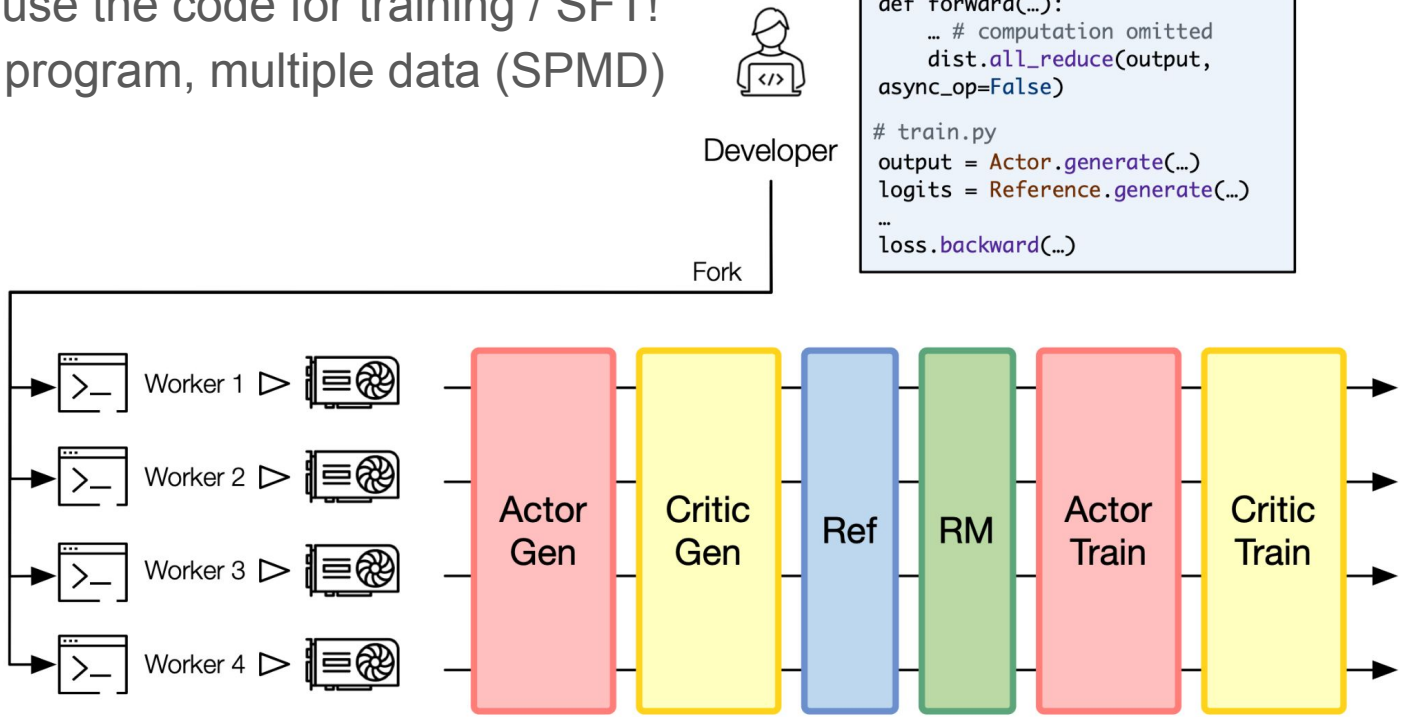

DeepSeek-V3 (671B)
RL with accuracy & format rewards
(RLVR)
1
DeepSeek-R1-Zero
SFT ("cold start") data
Train with "cold start" data
RL with accuracy, format, and consistency rewards
(RLVR)
SFT (CoT) data
SFT (knowledge) data
RL with rule-based verification (math, code) and human preference
(RLVR + RLHF)
2
DeepSeek-R1
Llama 3 & Qwen 2.5
3
DeepSeek-R1-Distill-Qwen (1.5B - 32B)
DeepSeek-R1-Distill-Llama (8B & 70B)

RL Systems: How do we program?

- Just reuse the code for training / SFT!
- Single program, multiple data (SPMD)

```
# models/llama.py
def forward(…):
    … # computation omitted
    dist.all_reduce(output,
async_op=False)

# train.py
output = Actor.generate(…)
logits = Reference.generate(…)
…
loss.backward(…)
```

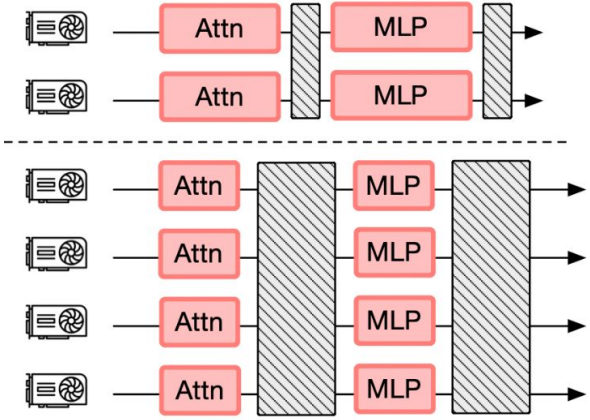


* DeepSpeed-Chat: Easy, Fast and Affordable RLHF Training of ChatGPT-like Models at All Scales

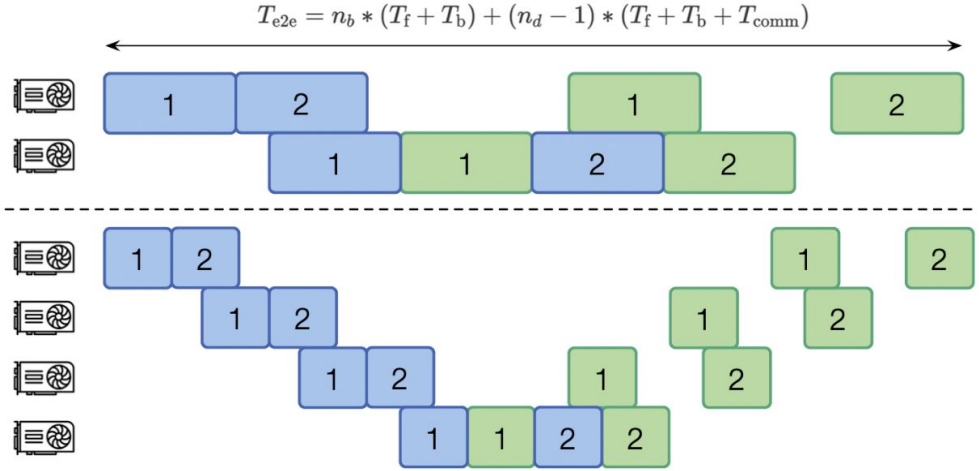
SPMD can be suboptimal!

Applying static parallelism to all models causes **over-parallelization**

- 2x the degree of model parallelism may not result in 2x speedup
- May be even slower or cause lower utilization



(1) Tensor Parallelism

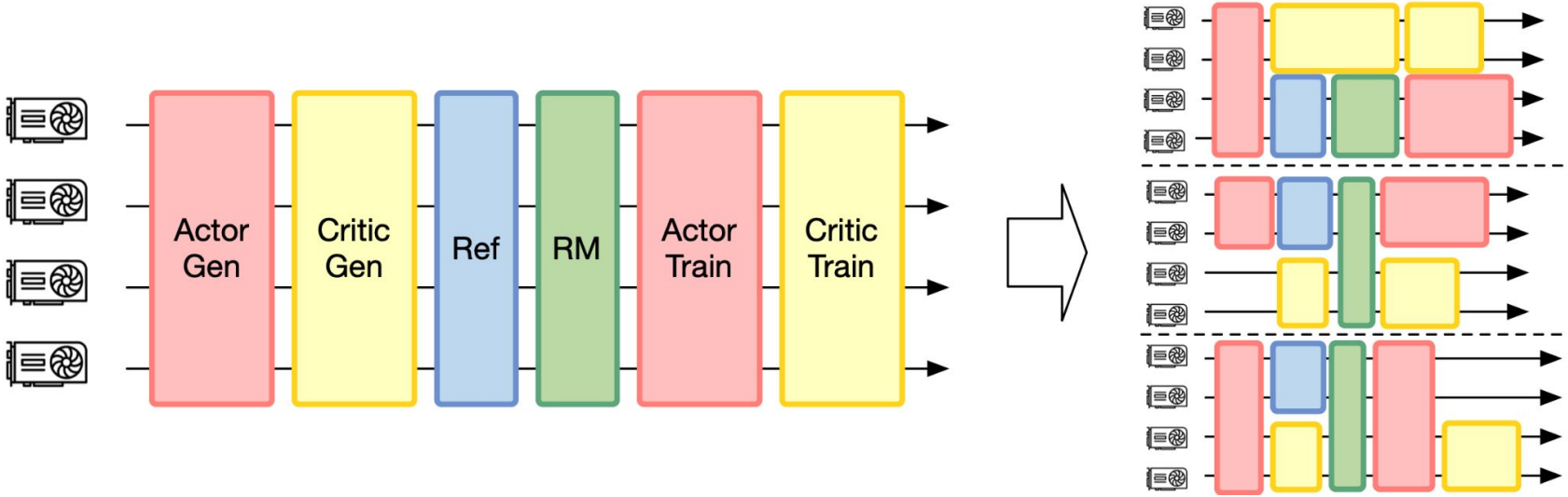


(2) Pipeline Parallelism

SPMD can be suboptimal!

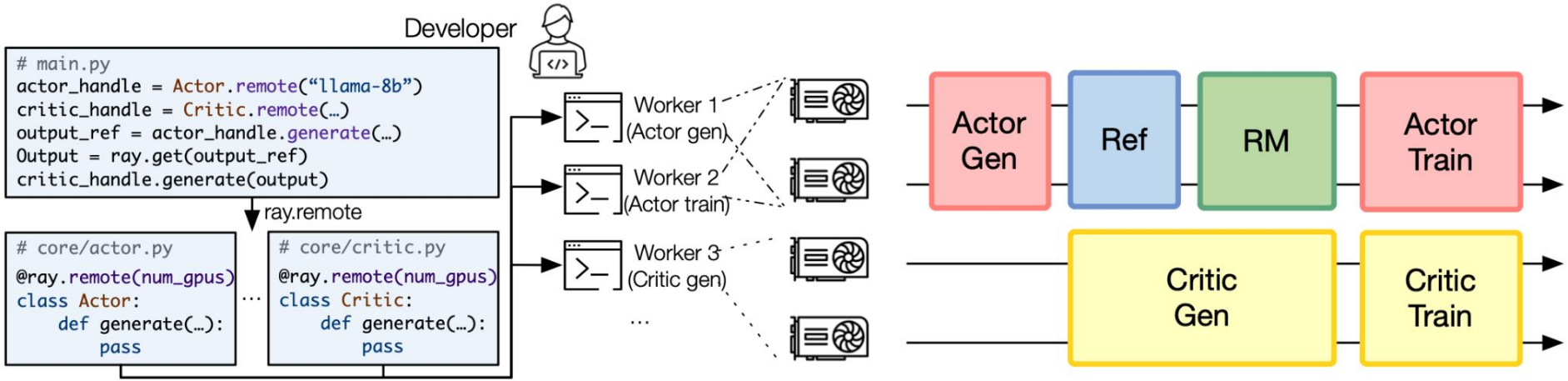
Applying static parallelism to all models causes **over-parallelization**

- We want to flexibly deploy different models on different set of GPUs based on their resource requirements!



Let’s go more distributed!

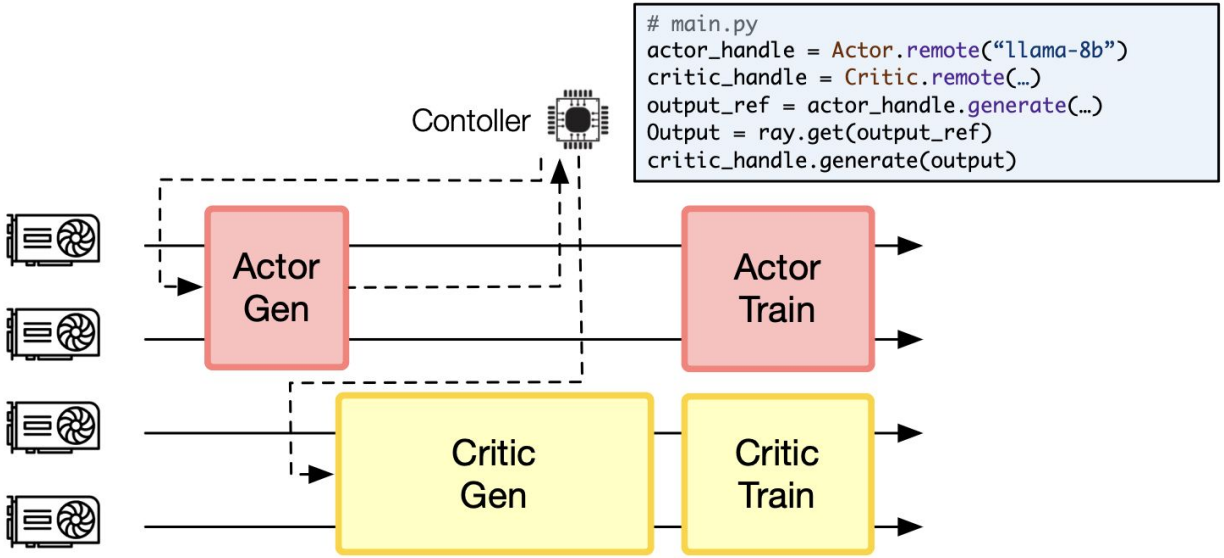
- First attempt: OpenRLHF + Ray
  - Single-controller architecture
- Dispatch different training stages to different processes and assign GPUs



* Ray: A Distributed Framework for Emerging AI Applications [OSDI’18]
* OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework [arxiv 2405.11143]

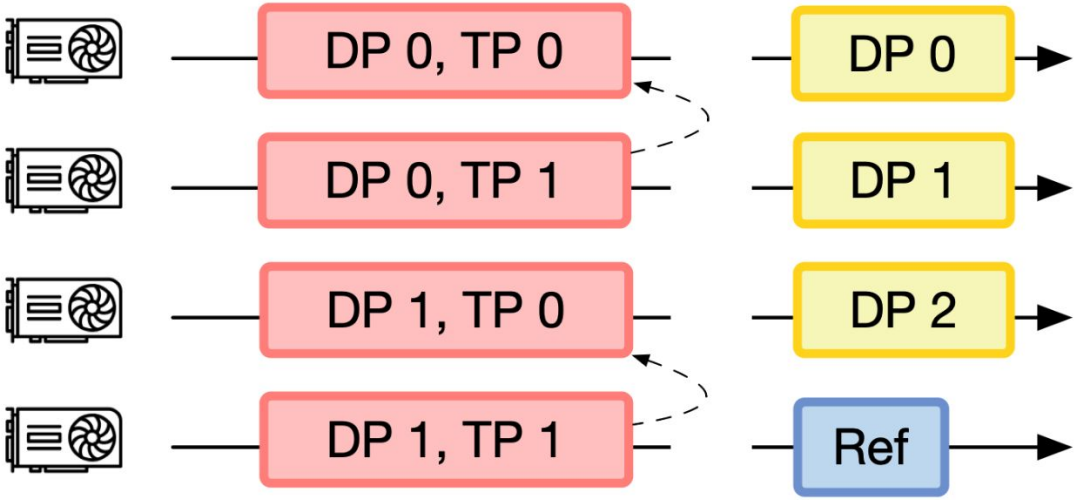
There’s a tradeoff between flexibility and performance…

- Indeed, single-controller / MPMD paradigm is flexible for developers to decide model implementation and placement
- But data movement implemented in the controller can be costly!



There’s a tradeoff between flexibility and performance…

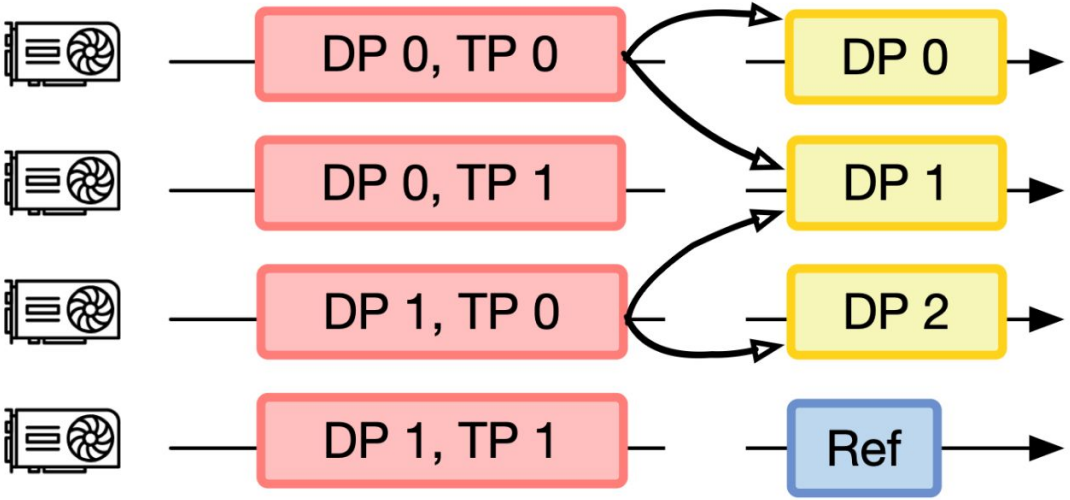
- Or, it can be inflexible…




```
# models/actor.py
def forward(…):
    … # computation omitted
    if tp_rank == 0:
        x_1 = recv(rank=1)
    else:
        send(rank=0, x)
```

There’s a tradeoff between flexibility and performance…

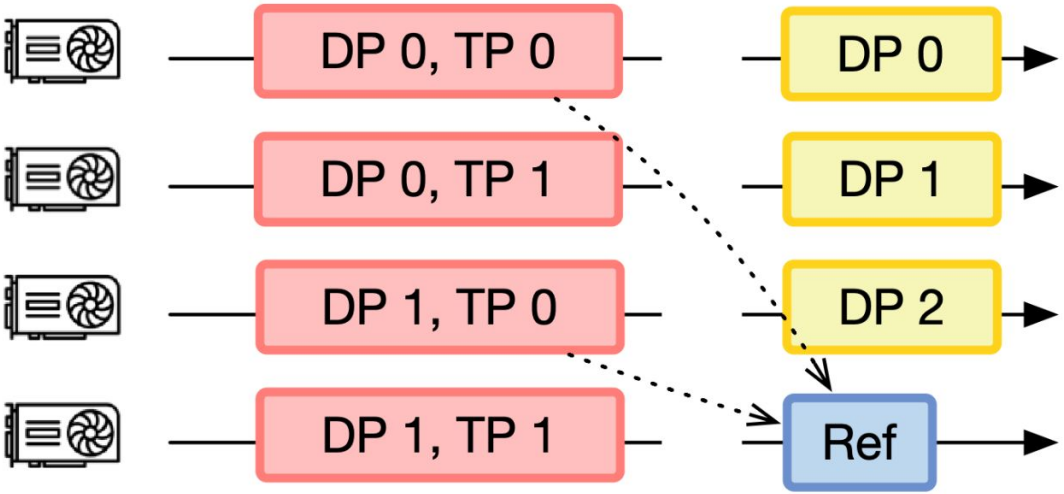
- Or, it can be inflexible…




```
# models/actor.py
def forward(…):
    … # computation omitted
    if tp_rank == 0:
        send(x[:bs//3*2],
group=“critic”,rank=0)
        send(x[bs//3*2:],…)
        …
# models/critic.py
def forward(…):
    if dp_rank == 0:
        actor_0 = recv(…)
        …
```

There’s a tradeoff between flexibility and performance…

- Or, it can be inflexible…




```
# models/ref.py
def forward(…):
    actor_0 = recv(…)
    actor_1 = recv(…)
    input = torch.cat(…)
    …
```

How to navigate the tradeoff?

- We should
  - Expose the single-controller semantics to user
  - Hide the communication details in the multi-controller implementation

How to tame the tradeoff?

- We should
  - Expose the single-controller semantics to user
  - Hide the communication details in multi-controller implementation
- Adopt a hybrid-controller framework!

Inter-Node: Single-Controller implementation

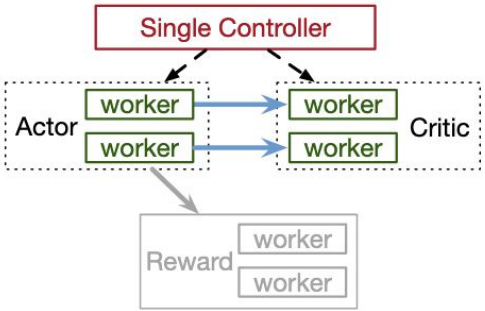
```
responses = actor.gen(prompts)
value = critic.comp_value(responses)
reward = rm.comp_reward(responses)
```

Intra-Node: Multi-Controller implementation

```
def gen(prompts):
  for _ in range(len):
    all_gather_weights()
    logit = model(prompts)
    ...
  return responses
```

```
def comp_values(res):
  all_gather_weights()
  values = model(res)
  return values
```

```
def comp_reward(res):
  all_gather_weights()
  reward = model(res)
  return reward
```



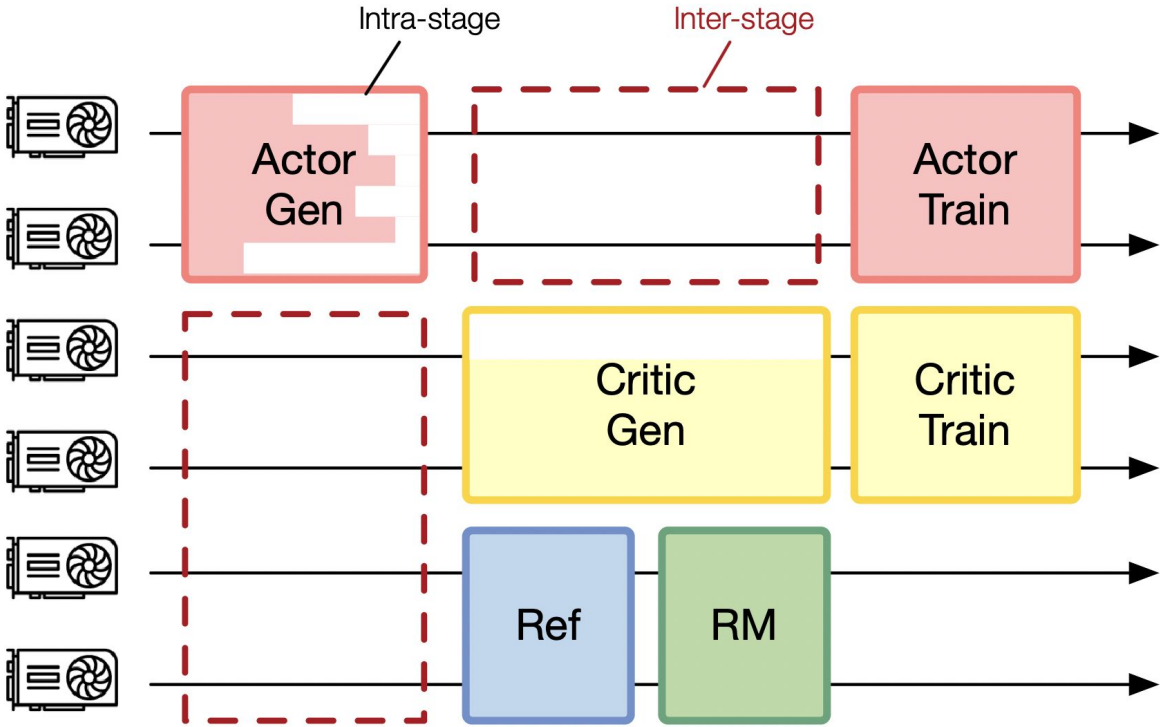
Flexible:
- Decouple the data and computation dependencies
- Seamless integrate any LLM systems. ✓

Efficient:
- Zero redundancy during transition
- Support different model placement strategies ✓

* HybridFlow: A Flexible and Efficient RLHF Framework [EuroSys'25]

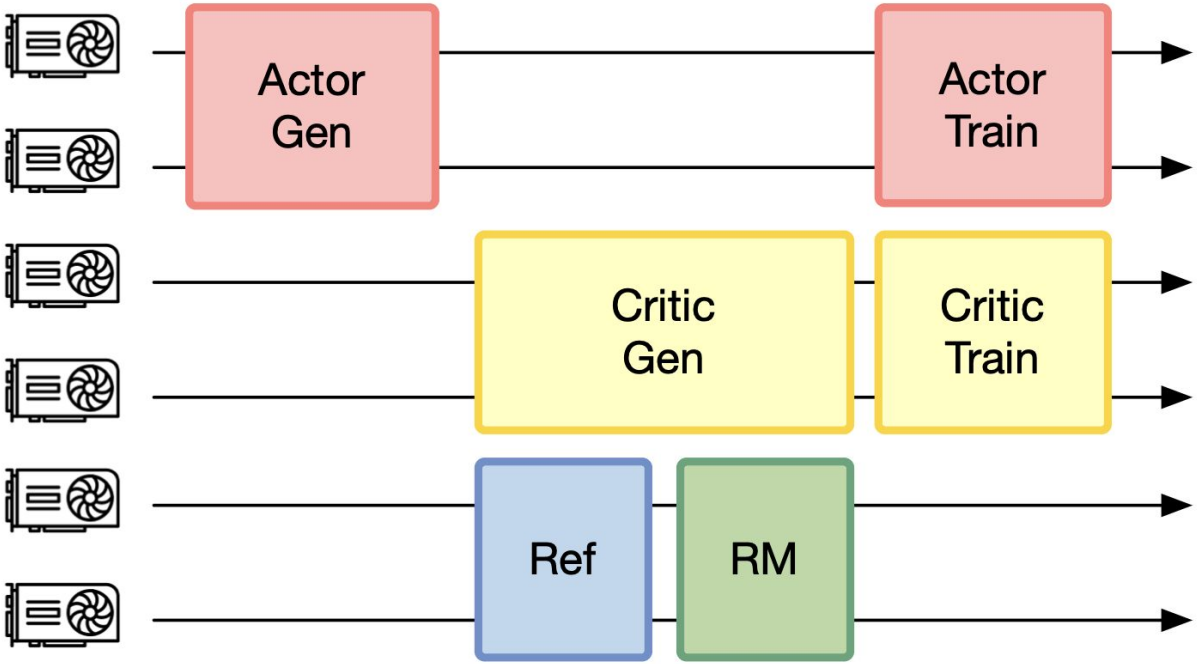
A large space for optimization!

- Intra / Inter-stage inefficiencies & opportunities



Optimization example

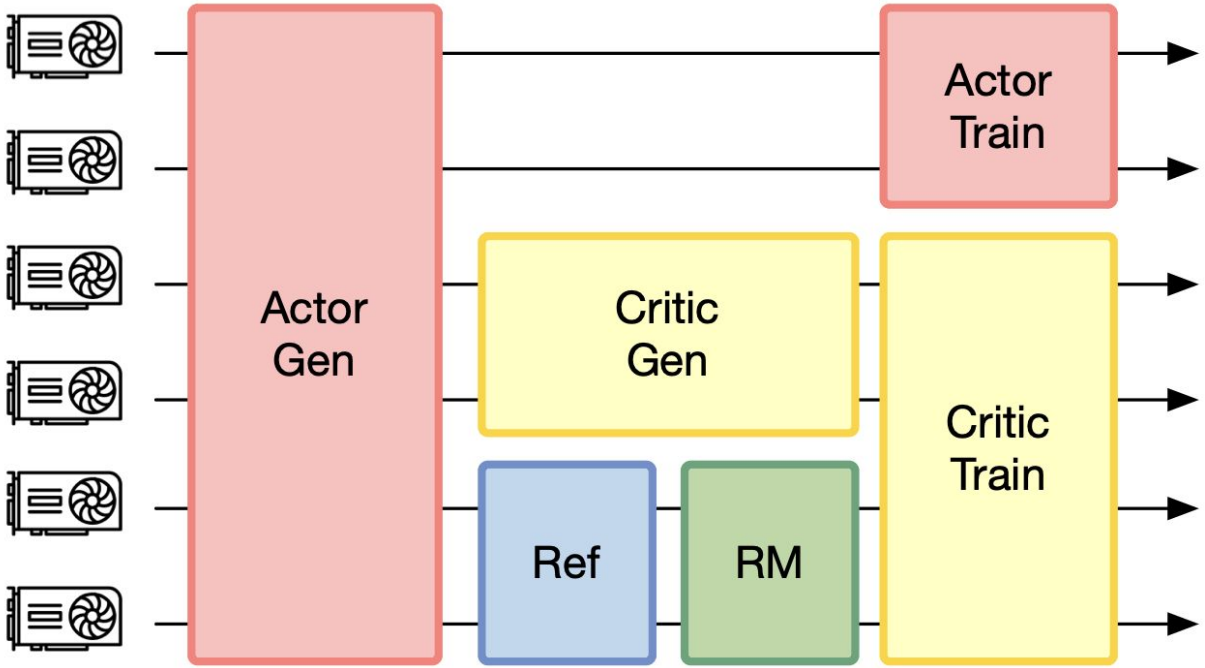
- Data dependency makes it hard to achieve optimal placement



Optimization example

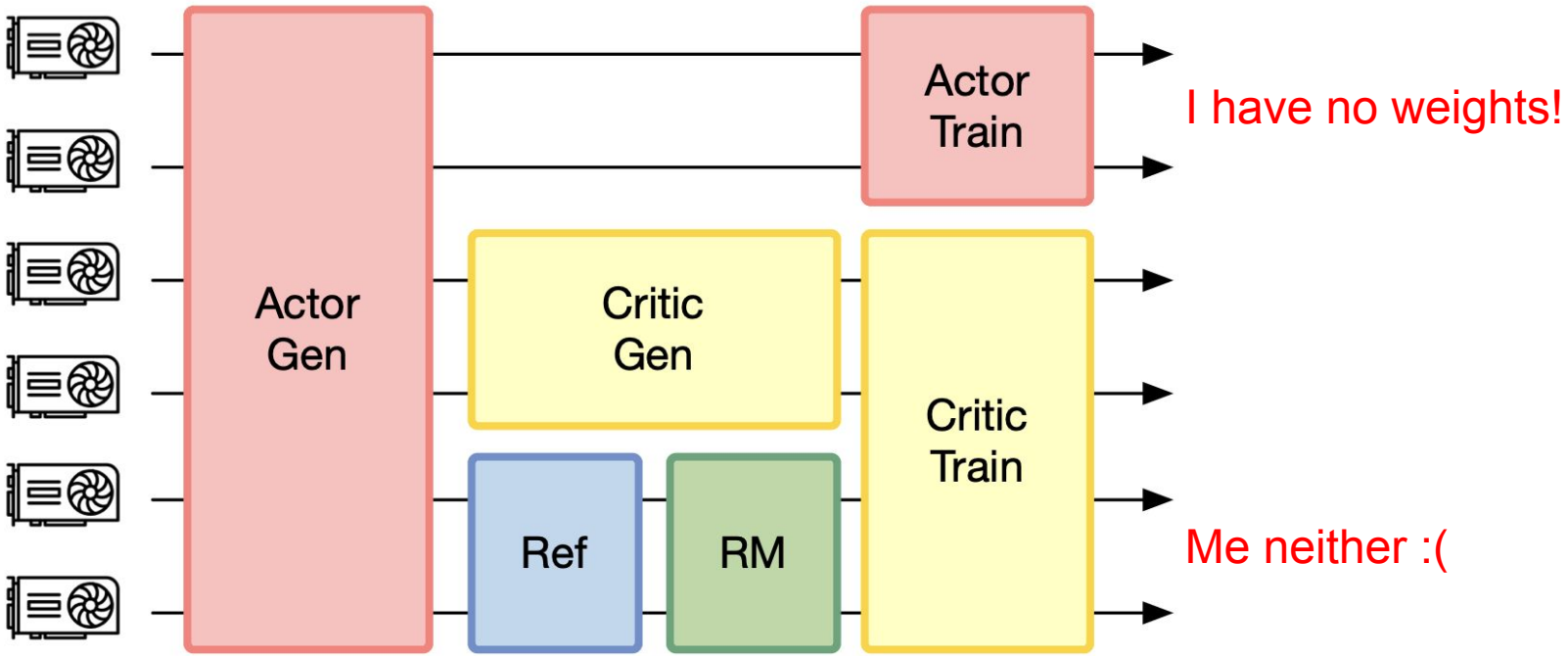
- Data dependency makes it hard to achieve optimal placement

Why not?



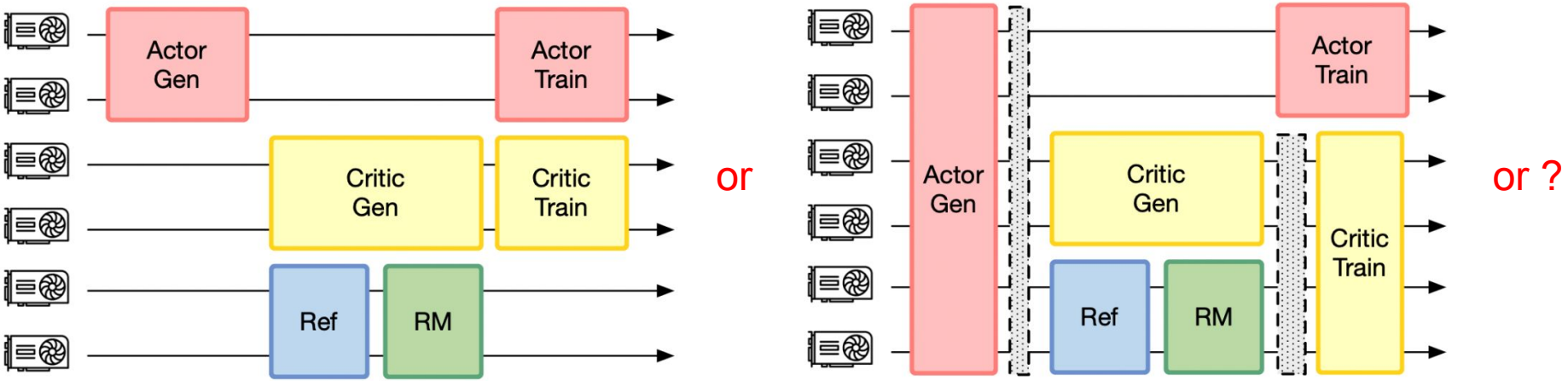
Optimization example

- Data dependency makes it hard to achieve optimal placement



Redistribute the models dynamically

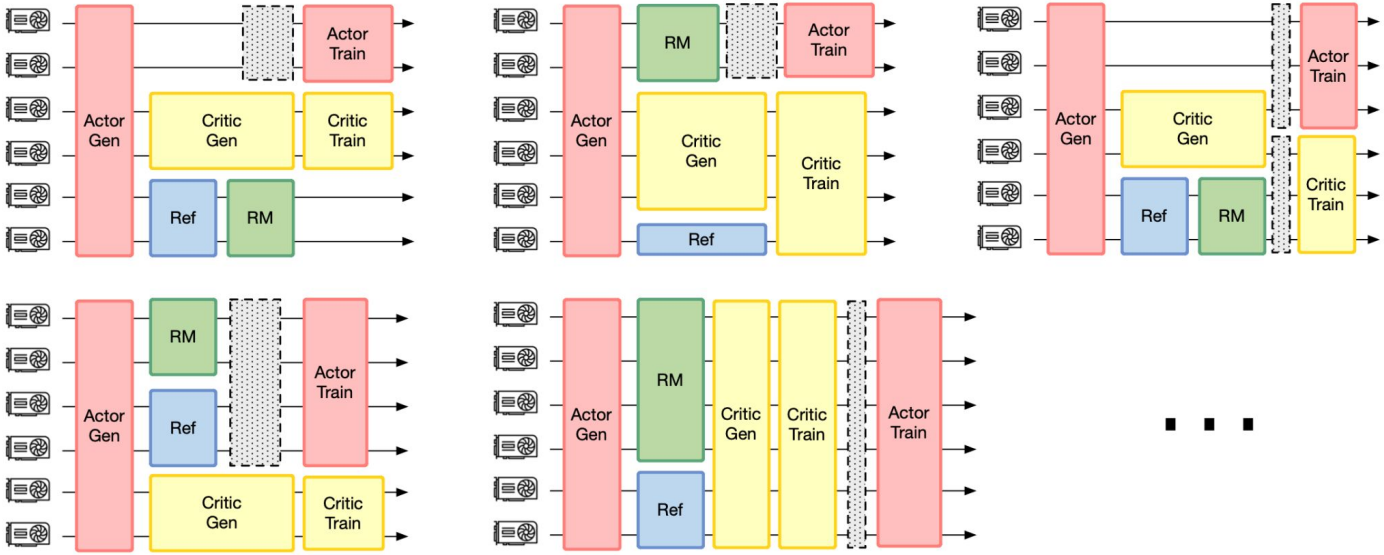
- Transfer overhead vs. compute throughput



* ReaL: Efficient RLHF Training of Large Language Models with Parameter Reallocation [MLSys'25]

Redistribute the models dynamically

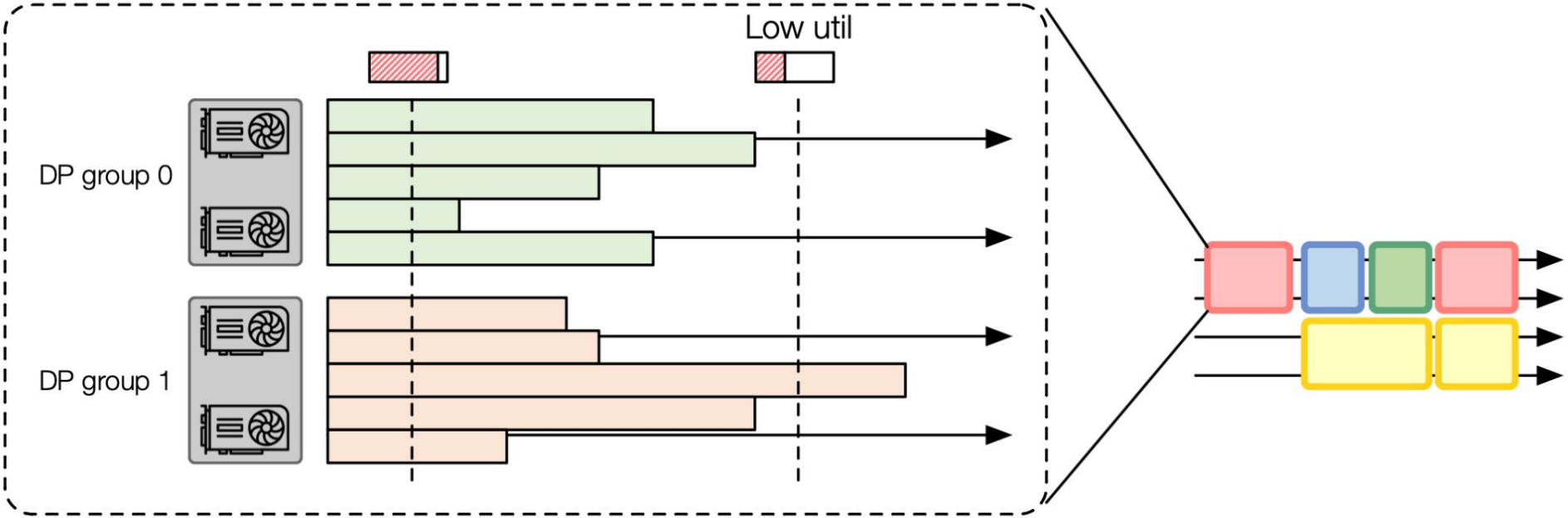
- Based on the data flow dependency, we can generate a lot of valid model placement schedules
- Use offline profiling data to **auto-search** the best schedule



* ReaL: Efficient RLHF Training of Large Language Models with Parameter Reallocation [MLSys'25]

Let's dive deeper!

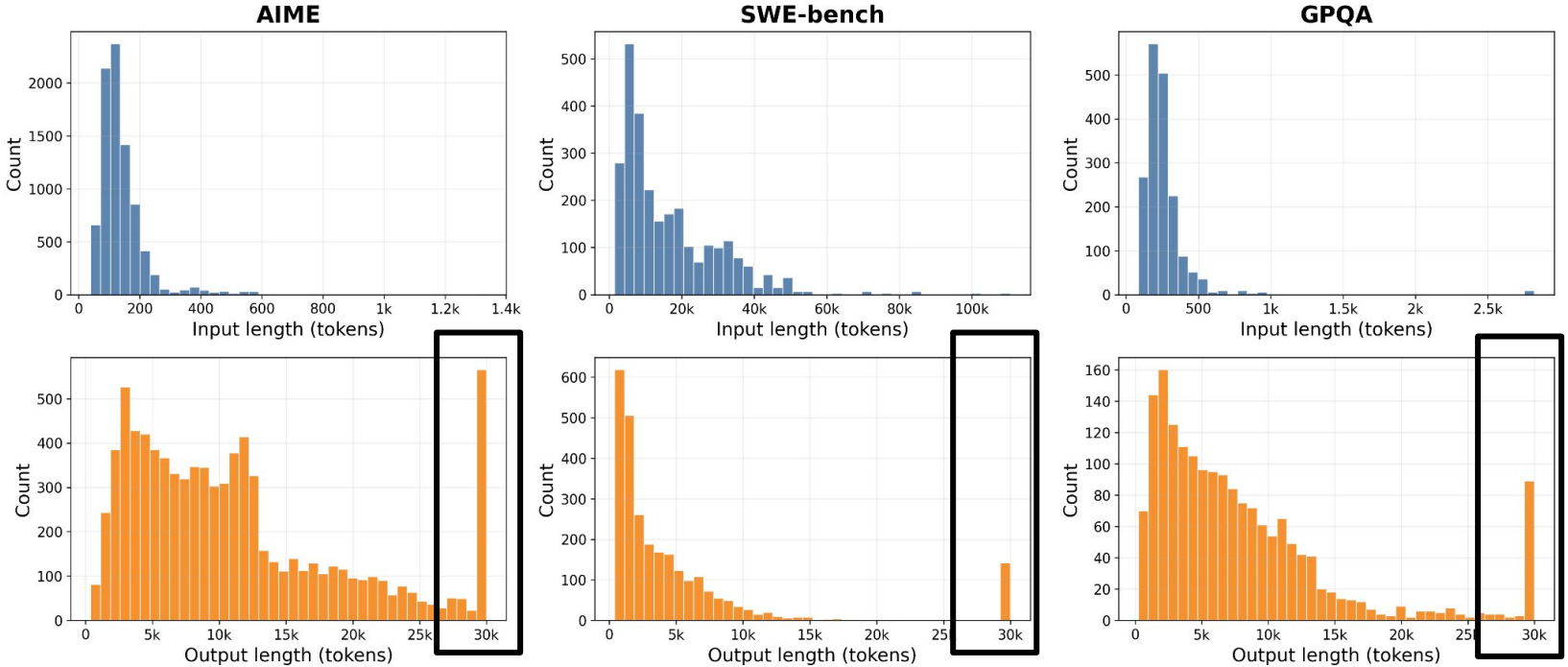
- Long tail problem in actor generation (rollout)
  - Single-batch offline inference is bottlenecked by the longest request



* Optimizing RLHF Training for Large Language Models with Stage Fusion [NSDI'25]

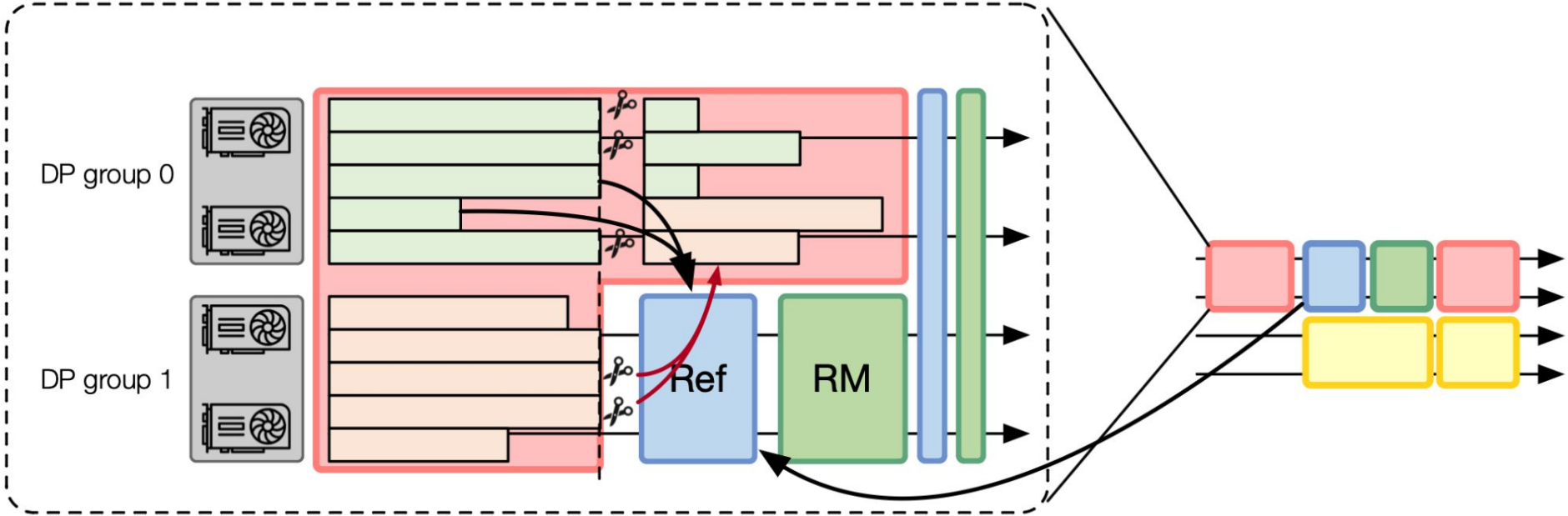
RL workload output length distribution

GLM 4.7 Model, max generation length set to 32K



Rollout is long and imbalanced

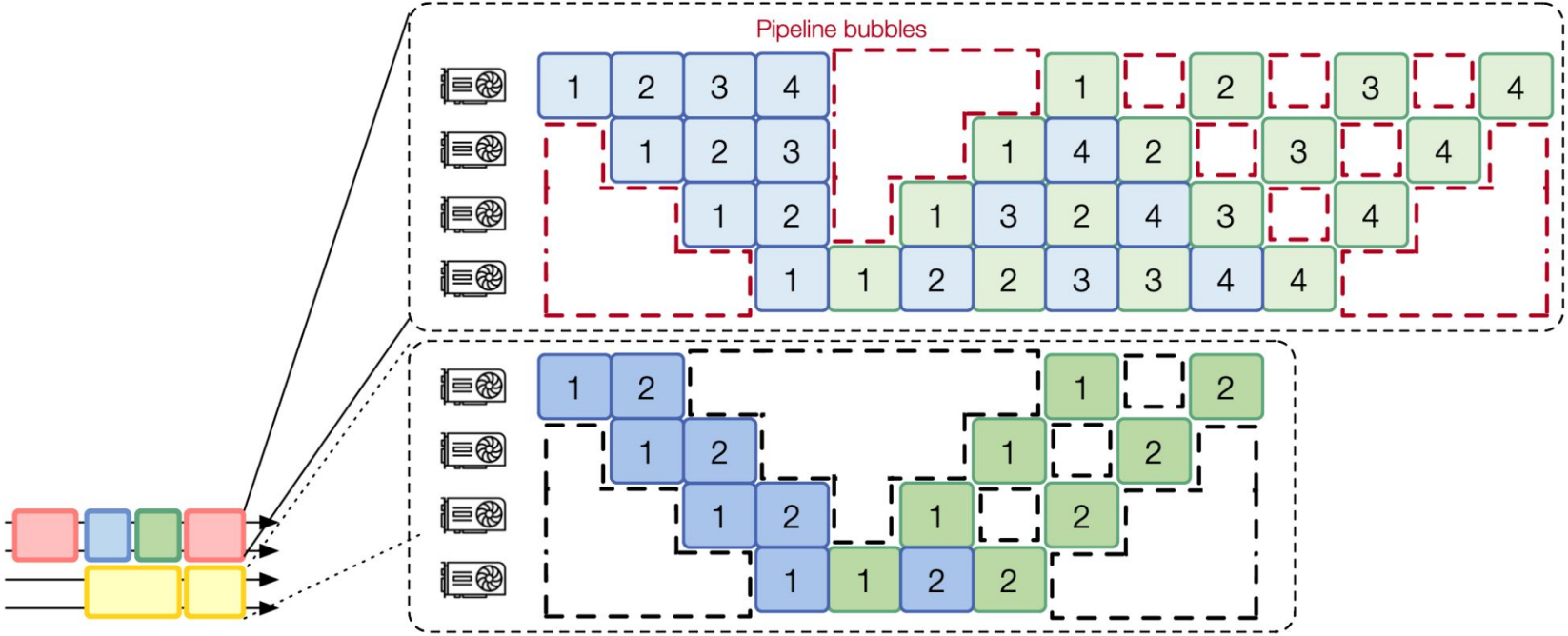
- We can fill in the bubbles by some of the later stages!



* Optimizing RLHF Training for Large Language Models with Stage Fusion [NSDI'25]

Similar ideas about multiple stages' co-scheduling…

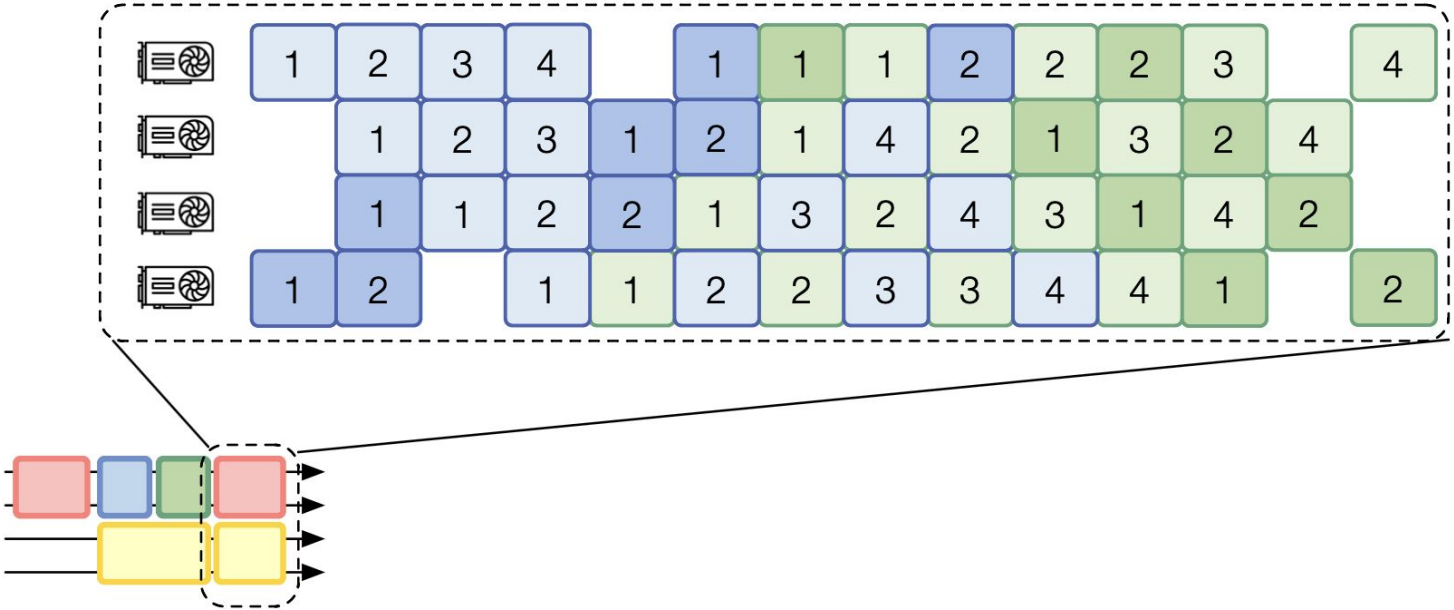
- Pipeline bubbles in actor and critic training



* Optimizing RLHF Training for Large Language Models with Stage Fusion [NSDI'25]

Similar ideas about multiple stages co-scheduling…

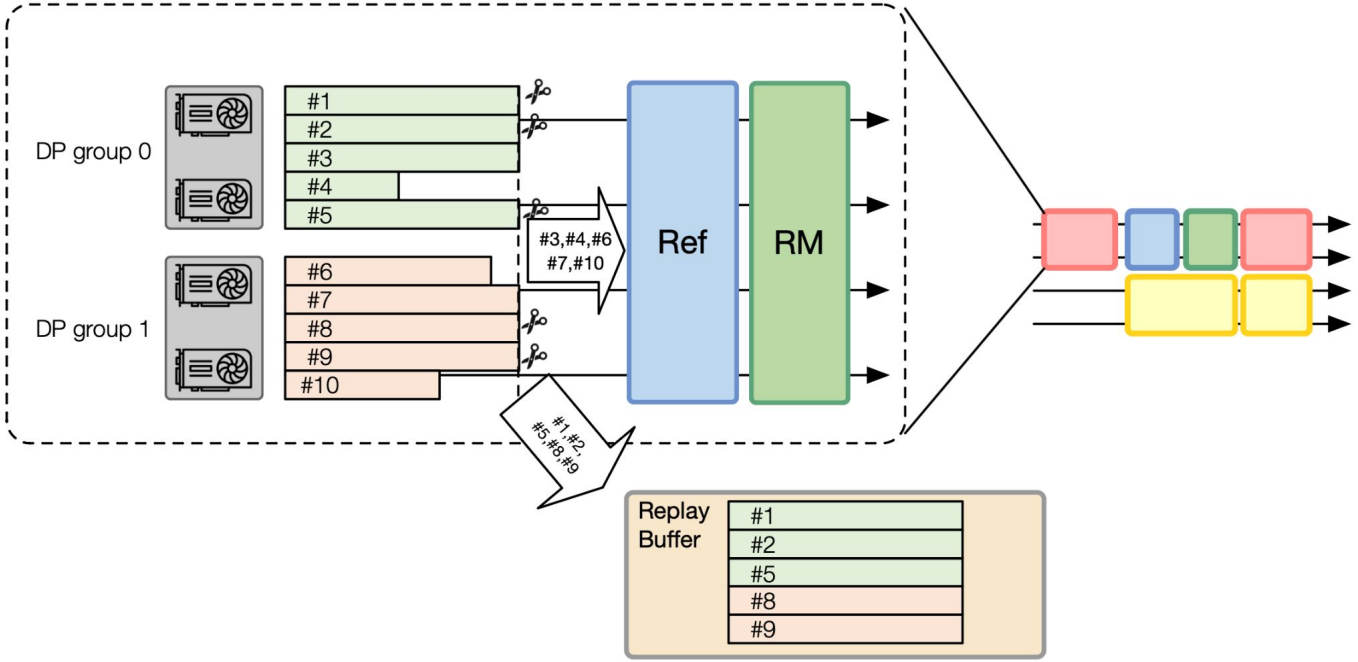
- Pipeline bubbles in actor and critic training



* Optimizing RLHF Training for Large Language Models with Stage Fusion [NSDI'25]

Same problem, different solutions

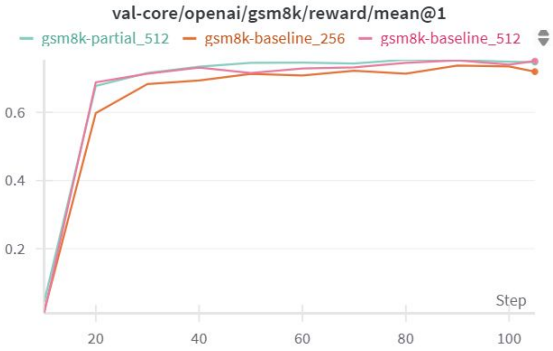
- We don’t have to finish all the job today (x

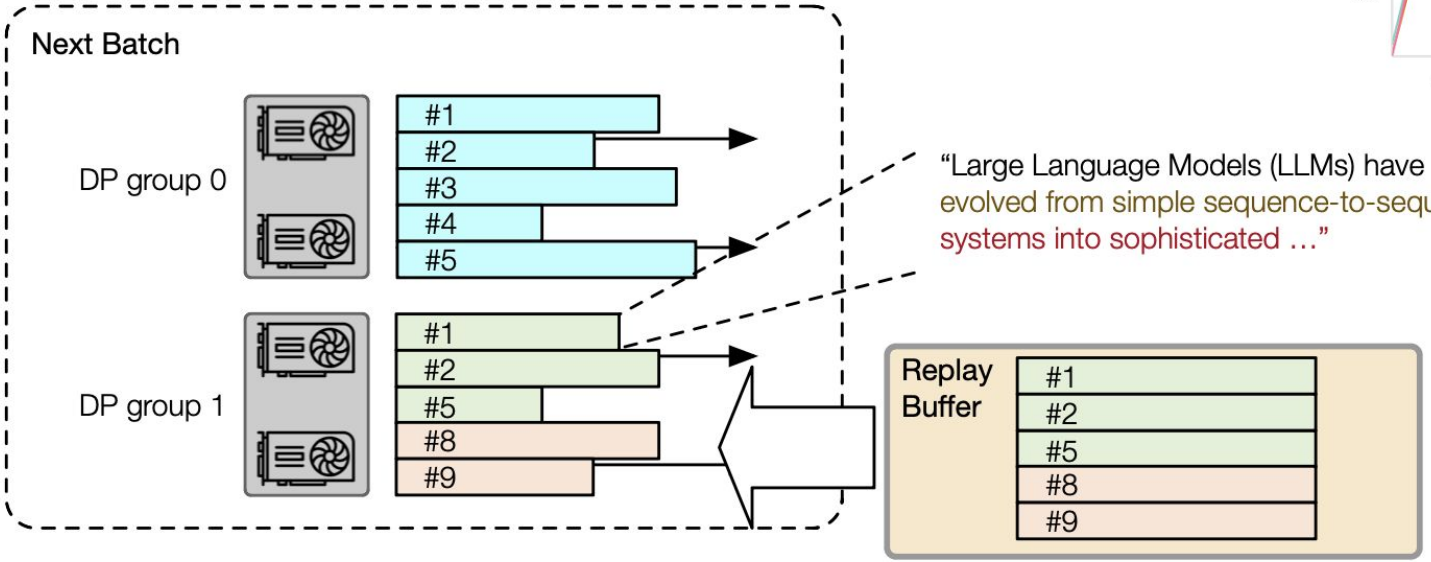


* Kimi k1.5: Scaling Reinforcement Learning with LLMs [arxiv 2501.12599]

Same problem, different solutions

- A request can be finished in different batches (weights)
  - Distributed system loves relaxed consistency :)
- Modify the algorithm, but the model quality is still good!



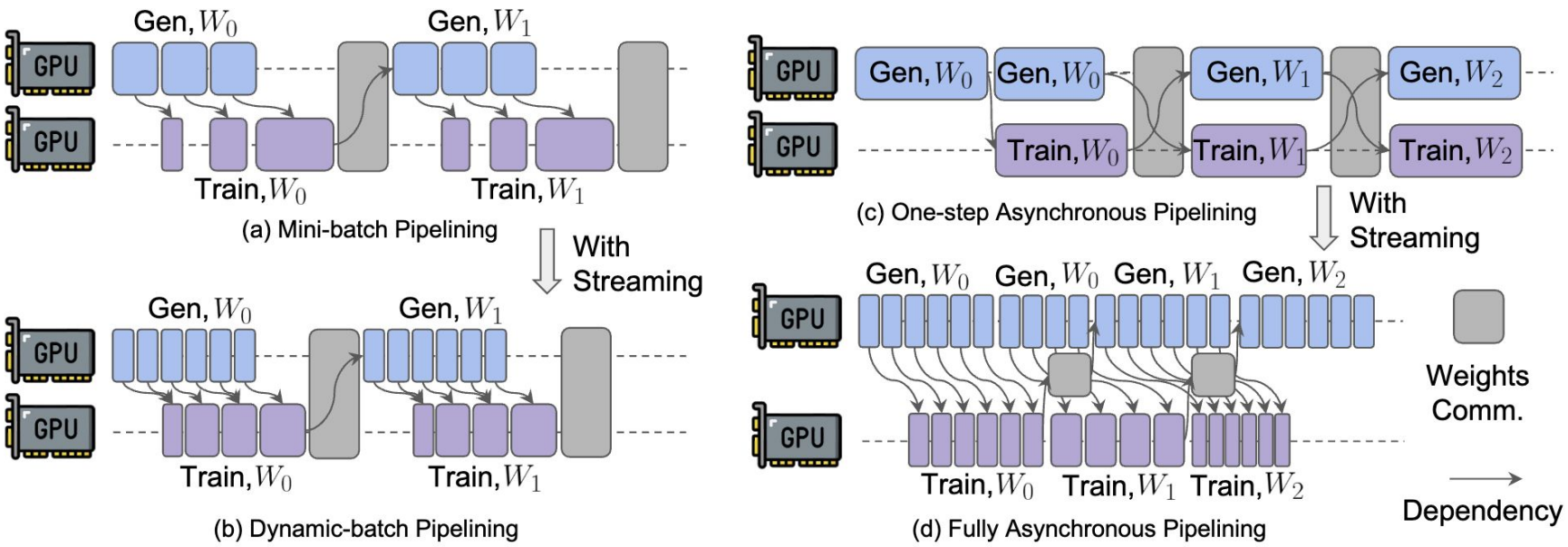


"Large Language Models (LLMs) have rapidly (prompt)
evolved from simple sequence-to-sequence (generated by old weight)
systems into sophisticated …" (generated by new weight)

* Kimi k1.5: Scaling Reinforcement Learning with LLMs [arxiv 2501.12599]

We can even go more aggressive!

- Inter-stage pipeline bubble
  - Use a fine-grained data flow to achieve **fully** async pipeline



* StreamRL: Scalable, Heterogeneous, and Elastic RL for LLMs with Disaggregated Stream Generation